

CALIFORNIA STATE UNIVERSITY NORTHRIDGE

LOW POWER ASIC DESIGN OF RISC CORE USING 90NM TECHNOLOGY

A graduate project submitted in partial fulfillment of
the requirements for the degree of Masters of Science
in Electrical Engineering

By:

Rajanish Donthi

December 2008

The graduate project of Rajanish Donthi is approved:

Dr. Xiaojun Geng

Date

Dr. Ali Amini

Date

Dr. Ramin Roosta, Chair

Date

ACKNOWLEDGEMENT

This project was an idea of Dr. Roosta for California State University Northridge students to take an initiative in the field of ASIC Design. This project is the first step ever taken in the ASIC field. I hope that it will be a good starting point from where, students can learn the basics of Front End ASIC Design Flow, Back End Flow and low power techniques they will be able to move ahead of me and learn t and detailed techniques of Verification as well.

I would like to thank Dr. Roosta for such a great idea, for his constant support and encouragement given to me towards this project. I also thank Professor Dr. Ali Amini and Professor Dr. Xiaojun Geng (Ashley) for their valued attention and information. I would also like to thank few people from SYNOPSYS for their contribution to this project they are Mr. Rajiv Dave who helped with setting up libraries and Mr. Prasad Lakamsani who helped by providing design. I am also thankful to Ms. Tanya for installing necessary tools and libraries.

Most importantly, I like to thank my parents, Mr. D.Govinda Rajulu and Mrs. D.G Geetha for giving me an opportunity to come to USA to pursue a highly qualified technical education of Master's Degree that has not only improved my career but it has also helped me define my long term career goals. Special thanks to my best friends, Shilpa Jammula and Pradeep Jayaramu, helping me for successful completion of my project.

TABLE OF CONTENTS

Signatures.....	ii
Acknowledgement.....	iii
List of figures.....	iv
List of Tables.....	vi
ABSTRACT.....	xi
 CHAPTER: 1 - INTRODUCTION.....	 XIII
1.1 Application Specific Integrated Circuits and Systems.....	xiii
1.2 Importance of Low Power Design	xv
CHAPTER: 2 - LOW POWER DESIGN	XVIII
2.1 Need for Low Power Design.....	xviii
2.1.1 Design Flow with and without Power.....	xviii
2.2 Basic Concepts for Power.....	xx
2.2.1 Static Power	xxi
2.2.2 Dynamic Power.....	xxii
2.2.3 Short-Circuit Power	xxv
2.2.4 Leakage Power.....	xxv
2.2.5 Voltage drop effects.....	xxv
2.2.6 Electro migration effects.....	xxvii
2.3 Low Power Design Techniques	xxvii
2.3.1. Voltage Scaling.....	xxvii
2.3.2. Parallelism and Pipelining	xxviii
2.3.3. Device Enabling.....	xxxi
2.3.4. Interconnect Minimization.....	xxxiii
2.3.5. Logic Design Techniques	xxxiv
CHAPTER: 3 - ASIC DESIGN FLOW AND IMPLEMENTATION	XXXVI
3.1 ASIC Design Flow.....	xxxvi
3.2 ASIC Implementation	xxxvii
CHAPTER: 4 - DESIGN COMPILER.....	XL

4.1 What is Synthesis?	xlii
4.2 System Library Variables (Linked & Target Library)	xliii
4.2.1 Variables	xliii
4.2.2 Attributes	xliii
4.3 Specifying Libraries	xliv
4.3.1 Technology Libraries	xliv
4.3.2 Logic Library	xliv
4.3.3 Physical Library	xliv
4.3.4 Symbol Libraries	xl v
4.3.5 Design Ware Libraries	xl v
4.4 Design Flow	xl v
4.4.1 Read Design	xl v
4.4.2 Define Design Environment	xlvi
4.4.3 Design Constraints	xlix
4.4.4 Optimization Process	liii
4.4.5 Analyzing and Resolving Design Problems	liv
CHAPTER: 5 - IC COMPILER	LV
5.1 Design Objects	lviii
5.2 Preparing the Design	lix
5.3 Setting up the Libraries	lix
5.3.1 Setting up the Logic Libraries	lix
5.3.2 Setting up the Physical Libraries	lx
5.4 Setting Up the Power and Ground Nets	lxi
5.5 Annotating the Physical Data	lxi
5.5.1 Reading DEF Files	lxi
5.5.2 Reading Floorplan Files	lxi
5.5.3 Copying Physical Data	lxii
5.6 Preparing for Timing Analysis and RC Calculation	lxii
5.6.1 Setting up the TLU-Plus Files	lxiii
5.6.2 Setting Timing Constraints	lxiii
5.7 Placement	lxiv

5.8 Clock Tree Synthesis (CTS)	lxvi
5.8.7 Optimizing for Power	lxvii
5.8.8 Implementing the Clock Trees.....	lxix
5.9 Routing.....	lxxi
5.9.1 Prerequisites for Routing	lxxii
5.9.2 Routing and Postroute Optimization of Signal Nets.....	lxxii
5.9.3 Global Routing.....	lxxiii
5.9.4 Track Assignment	lxxiv
5.9.5 Detail Routing and Search and Repair.....	lxxiv
5.10 Power Optimization in ICC	lxxiv
5.10.1 Low-Power Placement.....	lxxv
5.10.2 Leakage Power Optimization.....	lxxvi
5.10.3 Leakage Power Using Multithreshold Voltage Cells.....	lxxvii
5.10.4 Dynamic Power Optimization.....	lxxviii
5.10 Physical Verification.....	lxxx
5.12 Fabrication	lxxxix
CHAPTER: 6 - RISC ARCHITECTURE.....	LXXXII
6.1 Features	lxxxii
6.2 Introduction.....	lxxxiii
6.3 OpenRISC 1200 Architecture Overview	lxxxiii
6.3.1 CPU/DSP	lxxxiv
6.3.2 Instruction unit.....	lxxxv
6.3.3 General-Purpose Registers.....	lxxxvi
6.3.4 Load/Store Unit.....	lxxxvi
6.3.5 Integer Execution Pipeline.....	lxxxvii
6.3.6 MAC Unit	lxxxviii
6.3.7 System Unit.....	lxxxviii
6.3.8 Exceptions.....	lxxxviii
6.4 Register Set	lxxxix
6.4.1 Features	lxxxix
6.4.2 General-Purpose Registers (GPRs).....	lxxxix

6.5 RISC Design Implementation - Command line flow.....	xc
6.5.1 Design Compiler flow (Front-End).....	xc
6.5.2 IC Compiler command line flow	xciii
6.6 Results.....	xciv
6.6.1 Analysis.....	xciv
6.6.2 Timing and power characteristics	xciv
6.6.3 Power Optimization potential with clock gating	xcv
6.7 Conclusion	xcv
REFERENCES	XCVII
APPENDIX A.....	XCIX
APPENDIX B	CV

LIST OF FIGURES

Figure: 1.1 - Application specific integrated circuits (ASIC): (a) market share of different ASIC types [46]; (b) standard-cell-based ASIC design flow.....	xiv
Figure: 1.2 - History of the power consumption and the complexity of INTEL'S desktop processors: (a) absolute power and number of transistors per chip at the date of introduction; (b) power density at the date of introduction.	xvi
Figure: 2.1 - VLSI Design Flow	xx
Figure: 2.2 - Static Leakage Currents	xxii
Figure: 2.3 - Switching Power	xxiv
Figure: 2.4 - Internal Power	xxiv
Figure: 2.5 - A chain of inverters connected to the same power and ground tracks [9].	xxv
Figure: 2.6 - Energy product delay of a typical CMOS device	xxviii
Figure: 2.7 - Parallel implementation of a circuit	xxix
Figure: 2.8 - Pipelined implementation of a circuit	xxx
Figure: 2.9 - Scheduling.....	xxxii
Figure: 2.10 - Clock gating example.....	xxxii
Figure: 2.11 - Module interconnection using global bus and local bus.	xxxiii
Figure: 3.1 - ASIC Design Flow	xxxvi
Figure: 4.1 - Design Compiler	xl
Figure: 4.2 - Design Compiler Flow	xli
Figure: 4.3 - Synthesis in DC.....	xliii
Figure: 4.4 - Basic Design Environment.....	xlvi
Figure: 4.5 - Design Constraints for synthesis.....	xlix
Figure: 4.6 - Specification of the Input Delay	li
Figure: 4.7 - Specification of the Output Delay.....	lii
Figure: 4.8 - DC to ICC	liv
Figure: 5.1 - IC Compiler Design Flow	lvi
Figure: 5.2 - ICC Overview	lxiv
Figure: 5.3 - Iterative placement and optimization.....	lxv
Figure: 5.5 - Grid-Based Routing System.....	lxxi
Figure: 5.6 - IC Compiler Placement.....	lxxvi

Figure: 5.7 - Usage Model: Leakage Power Optimization	lxxviii
Figure: 5.7 - Usage Model: Dynamic Power Optimization	lxxix
Figure: 5.8 - Leakage Vs Dynamic Power	lxxx
Figure: 6.1 - Core's Architecture.....	lxxxiv
Figure: 6.2 - CPU/DSP Block Diagram	lxxxv
Figure: A.1 - Layout view before Floor plan.....	xcix
Figure: A.2 - Layout view after Floor plan.....	c
Figure: A.3 – Layout view after placement	ci
Figure: A.4 - Layout view after placement with critical path.....	cii
Figure: A.5 - Standard cell view after routing	ciiii
Figure: A.6 - Schematic view of RISC core design in DC	civ

LIST OF TABLES

Table: 1 - Architecture summary.	xxxi
Table 2 – Stages of power optimization in ICC.....	lxxx
Table: 6.1 - Execution Time of Integer Instructions.....	lxxxvii
Table: 6.2 - General-Purpose Registers	lxxxix
Table: 6.3 - Timing and power consumption before and after place & route.....	xcv
Table: 6.4 - Power consumption before and after clock gating	xcv

ABSTRACT
LOW POWER ASIC DESIGN OF RISC CORE USING 90NM TECHNOLOGY

By

Rajanish Donthi

Master of Science in Electrical Engineering

In the past few decades with advancement in VLSI technology, Application Specific Integrated circuit (ASIC) chip density has increased and ASIC devices now provide a large number of smaller feature size transistors and can support higher clock speeds. While this advancement is beneficial for implementing larger and faster designs within a single chip, it also leads to increased power consumption. With the remarkable growth of ASIC based battery-powered systems, such as personal computing devices, wireless equipment, space-borne systems, and consumer electronics, low power ASIC design is of increased importance.

In this Project, investigate various low power ASIC design techniques to minimize dynamic power consumed by ASICs. The objective of this research is to minimize the power drawn by a design without altering its functionality and with minimal or no impact on its timing. The work focuses on the physical implementation of highly complex pattern based RISC design.

Reduction of power consumption of an ASIC design is attempted at the following design stages: high-level synthesis, mapping and placement and routing stage. Various low power design techniques and algorithms are discussed and applied to RISC core and has been shown that power consumed by the proposed low power design is significantly lower than a traditional design. The central idea behind all the design techniques proposed is to power optimize that is reduce dynamic and static power consumption. The techniques and algorithms presented in this project is automated and incorporated in an existing ASIC design flow.

CHAPTER: 1 - INTRODUCTION

1.1 Application Specific Integrated Circuits and Systems

The integrated circuit (IC) market is divided into the standard product (SP) segment and the application specific integrated circuit (ASIC) segment. Standard products are produced in large volumes and can be used in a variety of applications. Major advantages of using existing SPs are relatively low cost, off-the-shelf availability, and proven reliability. On the other hand, SPs are not optimized for any specific application and are, thus, usually inefficient regarding the performance, the power consumption, and the area. Application specific integrated circuits are optimized for a specific application or application domain. Therefore, ASICs provide performance-, power- and area-efficient implementations of specific functionalities. The efficiency, however, comes at the cost of longer design times, a larger potential for system failure, and a higher price per unit. About 98% of all ASICs are digital circuits fabricated in mainstream complementary metal oxide semiconductor (CMOS) technologies [19].

Application specific integrated circuits are designed using full-custom or semi-custom design styles. In full-custom design, every single transistor is optimized individually. This is an expensive design style seldom used for designing digital ASICs; full-custom ASICs accounted for only 19% of the total ASIC market in 2002 (see Figure 1.1a). The distinct characteristic of semi-custom design is the use of pre-designed or pre-fabricated building blocks. The important semi-custom ASIC types are standard-cell-based ICs, gate arrays, and programmable logic devices (PLD). Most digital ASICs rely on the standard cell concept (54% in 2002; expected to reach 61% by 2007; see Figure 1.1a). In the standard cell-based design style a variety of pre-designed logic gates, flip-flops, latches, half and full adders, etc. are available in so-called standard cell libraries. The major advantages of the standard cell concept over pre-fabricated gate arrays and PLDs are a higher integration density and a better performance. Compared with full-custom design, the standard cell concept raises the level of abstraction, which significantly reduces the design times and facilitates the development of tools for automated design and optimization. In recent years, robust and largely automated methodologies for the design

of standard-cell-based ASICs have been established, which is an important reason for the increasing use of ASICs in all kinds of electronic applications.

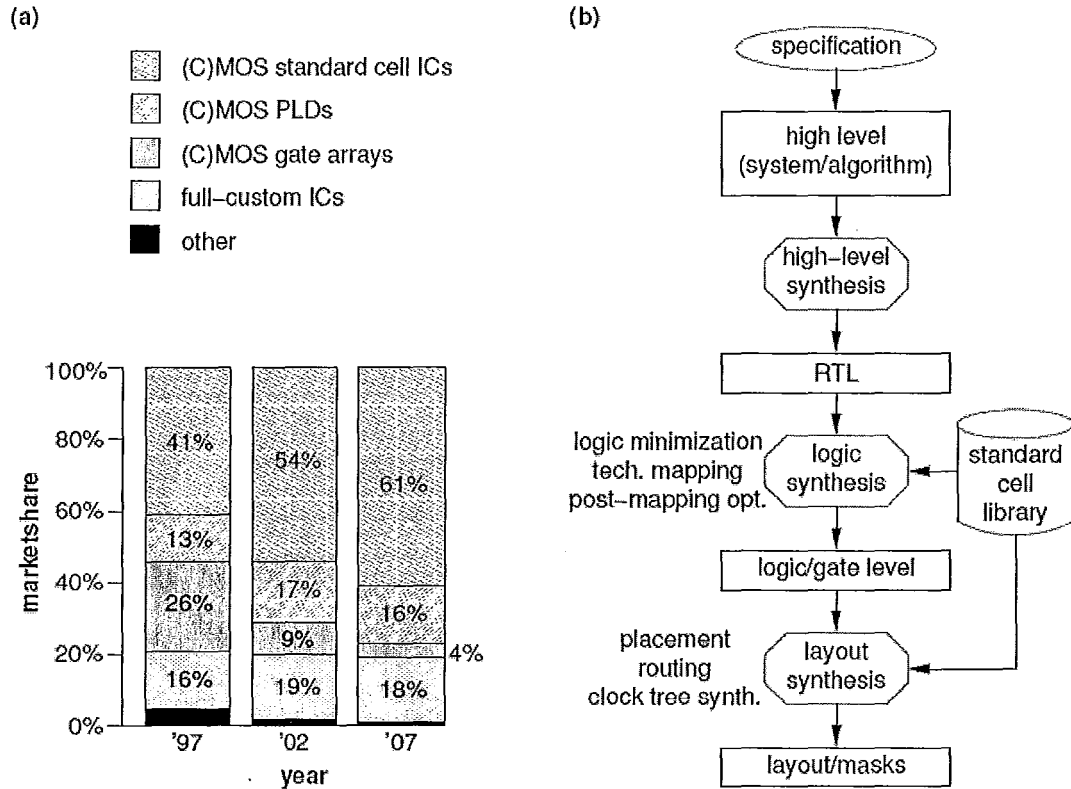


Figure: 1.1 - Application specific integrated circuits (ASIC): (a) market share of different ASIC types [46]; (b) standard-cell-based ASIC design flow

A typical standard-cell-based ASIC design flow is depicted in Figure 1.1b. The different levels of the design hierarchy are clearly visible. The design follows a top-down approach and starts with a specification of the required functionality and constraints. The specification is then translated into a high-level description written in a programming or hardware description language (HDL). This high-level model serves two purposes. Firstly, the specification can be verified and different algorithms providing the same function can be evaluated. Secondly, the model can be used as a reference for the verification of lower level design descriptions in subsequent phases of the design process. High-level synthesis is the task of generating a design description at the register transfer level (RTL) from the high-level model. Although both academic and commercial tools

exist, automated high-level synthesis has not yet become an integral part of industrial design flows. Thus, RTL modeling using HDLs such as VHDL (Very High Speed Integrated Circuit Hardware Description Language), Verilog, or SystemC still has to be done manually. The remaining design steps are largely automated. The RTL model is first mapped to a logic-/gate-level implementation using a logic synthesis tool and elements from a standard cell library. The result of this logic synthesis step is a technology dependent gate-level netlist. In the subsequent layout synthesis, the cells that appear in the netlist are (virtually) placed on the surface of the die, and the interconnections between the cells are routed.

Interconnect routing means that the exact shapes and locations of all wires are defined. The final result of the design process is a set of data providing all the information required for fabricating the production masks. The design flow described above is actually not restricted to ASIC design. In fact, as the quality of the design tools improves and as the maximum complexity of standard-cell-based designs increases, thus reducing the quality gap between full-custom and semi-custom designs, the standard cell concept is more and more adapted to the design of most digital parts of modern systems-on-a-chip (SoC), including embedded microprocessor, microcontroller and digital signal processor (DSP) cores. Even SPs are sometimes designed using standard cells instead of full-custom design styles in order to reduce the design time.

1.2 Importance of Low Power Design

There has been tremendous progress in semiconductor technology since the first ICs were introduced in the 1960's. The minimum feature size, i.e. the minimum dimension of the semiconductor structures, has become smaller and the die sizes have increased. Consequences of this technology scaling trend are reduced device capacitances, higher integration densities, performance improvements, and increased circuit complexities. In the past, the circuit performance and the chip area were the major issues in IC design. This has changed over the last ten years. The power consumption is now another major design criterion. This development has been driven mainly by the rapid growth of the portable consumer electronics market, where system running time, battery weight, and

battery volume are critical parameters. The aforementioned increase in integration density and circuit performance, however, has led to enormous on-chip power and power densities, as indicated by the two graphs in Figure 1.2. Since excessive total power and power density cause serious reliability problems, the power consumption is no longer a specific problem of mobile applications. In fact, it is equally critical, if not more, in the design of high-performance ICs for non-battery-powered applications.

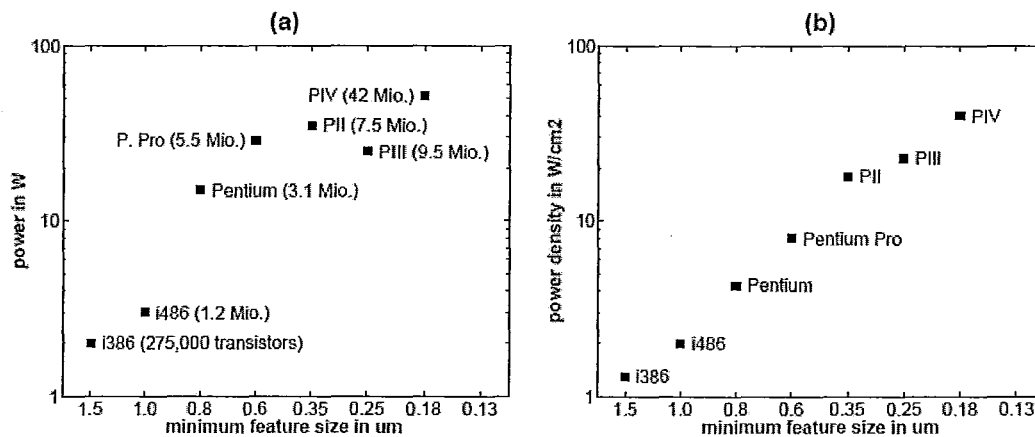


Figure: 1.2 - History of the power consumption and the complexity of INTEL'S desktop processors: (a) absolute power and number of transistors per chip at the date of introduction; (b) power density at the date of introduction.

The evolution of the battery technology is slow compared with the progress in semiconductor technology, so that the developers of portable applications cannot count on significant improvements in this field as a solution to the running time issue. The power consumption of existing portable or non-portable applications can be reduced by transferring selected ICs to a next-generation fabrication technology. Provided that the functionality and the performance of the ICs remain unchanged, the technology-scaled implementations dissipate less power because of smaller circuit capacitances and lower supply voltages. On the other hand, when technology scaling is exploited for maximizing the circuit complexity and the performance in order to meet the requirements of new applications, the power consumption actually increases, as discussed in the preceding paragraph. Ceramic packages, cooling fans, or other sophisticated cooling means may be used for removing the heat from such complex high performance ICs, so as to guarantee

reliability. Expensive external cooling means, however, increase the overall system cost and should, therefore, be avoided. It follows from the above arguments that neither improvements in battery or semiconductor technology nor external.

CHAPTER: 2 - LOW POWER DESIGN

2.1 Need for Low Power Design

In the early 1970's designing digital circuits for high speed and minimum area were the main design constraints. Most of the EDA tools were designed specifically to meet these criteria. Power consumption was also a part of the design process but not very visible. The reduction of area of digital circuits is not as big issue today because with new IC production techniques, many millions of transistors can be fit in a single IC. However, shrinking sizes of circuits have paved the way for reduced power consumption in order to have an extended battery life. Also in submicron technologies, there is a limitation on the proper functioning of circuits due to heat generated by power dissipation. Market forces are demanding low power for not only better life but also reliability, portability, performance, cost and time to market. This is very true in the field of personal computing devices, wireless communications systems, home entertainment systems, which are becoming popular now-a-days. Devices that are also used for high-performance computing particularly need to dissipate less power to function correctly and for a long period of time.

Keeping all these in mind, low power design has become one of the most important design parameters for VLSI (Very Large Scale Integration) systems.

2.1.1 Design Flow with and without Power

A top-down ordinary VLSI design approach is illustrated in Figure 2.1. The figure summarizes the flow of steps that are required to follow from a system level specification to the physical design. The approach was aimed at performance optimization and area minimization. However, introducing the third parameter of power dissipation made the designers to change the flow as you shown in the right-hand side of the Figure 2.1.

In each of the design levels are two important power factors, namely power optimization and power estimation. Power optimization is defined as the process of obtaining the best design knowing the design constraints and without violating design specifications. In order to meet the design and required goal, a power optimization technique unique to that

level should be employed. Power estimation is defined as the process of calculating power and energy dissipated with a certain percentage of accuracy and at different phases of the design process. Power estimation techniques evaluate the effect of various optimizations and design modifications on power at different abstraction levels. Generally a design performs a power optimization step first and then a power estimation step, but within a certain design level there is no specific design procedure. Each design level includes a large collection of low power techniques. Each may result in a significant reduction of power dissipation. However, a certain combination of low power techniques may lead to better results than another series of techniques.

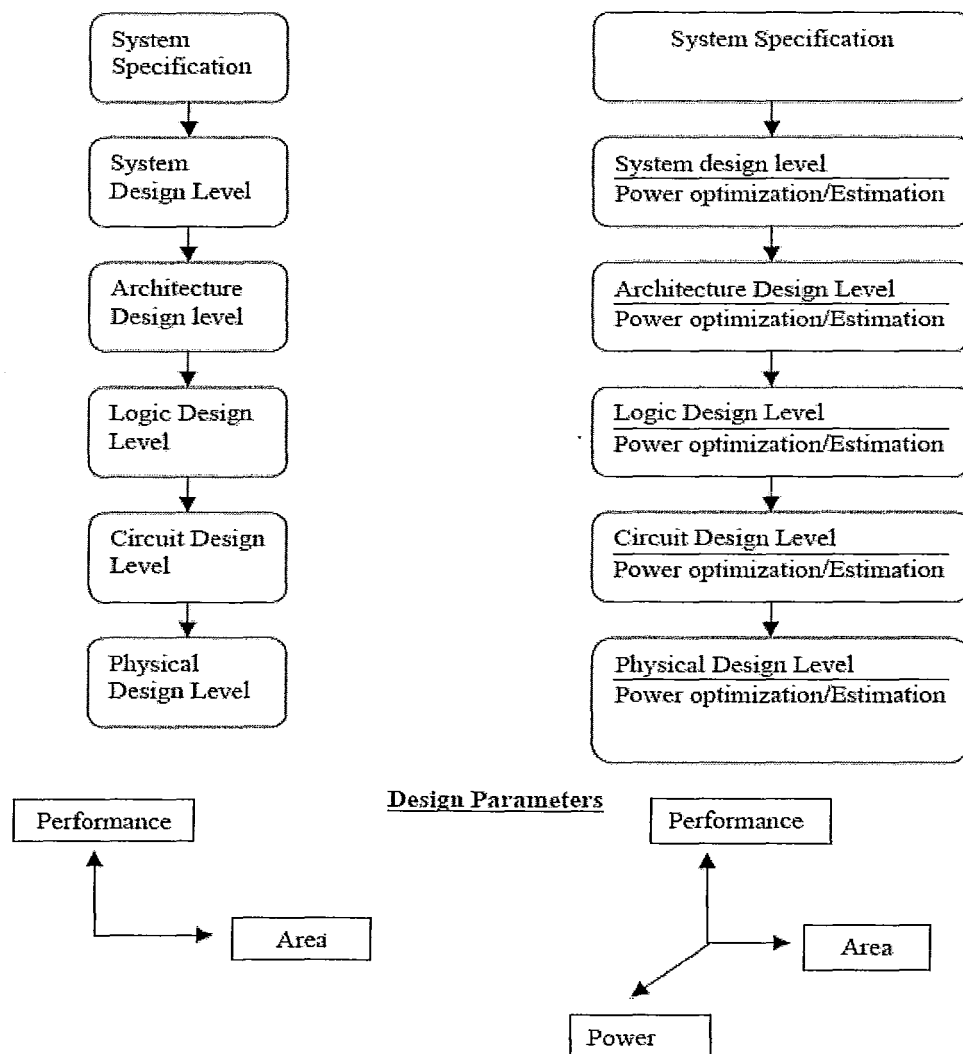


Figure: 2.1 - VLSI Design Flow

Generally, dynamic power is consumed when capacitors in the circuits are either charged or discharged due to switching activities. So at higher levels of a system this power dissipation is conserved by reducing the switching activities which is done by shutting down portions of the system when they are not needed. Large VLSI circuits contain different components like a processor, a functional unit and controllers. The idea of power reduction is to stop any of the components of the processor when they are not needed so that less power will be dissipated when the processor is operating.

2.2 Basic Concepts for Power

The power dissipation of digital CMOS circuits can be described by

$$P_{avg} = P_{dynamic} + P_{short-circuit} + P_{leakage} + P_{static}$$

P_{avg} is the average power dissipation, $P_{dynamic}$ is the dynamic power dissipation due to switching of transistors, $P_{short-circuit}$ is the short-circuit current power dissipation when there is a direct current path from power supply down to ground, $P_{leakage}$ is the power dissipation due to leakage currents, P_{static} and is the static power dissipation.

2.2.1 Static Power

Static power is the power dissipated by a gate when it is not switching that is, when it is inactive or static. Ideally, CMOS (Complementary Metal Oxide Semiconductor) circuits dissipate no static (DC) power since in the steady state there is no direct path from V_{dd} to ground. This scenario can never be realized in practice, since in reality the MOS transistor is not a perfect switch. There will always be leakage currents, subthreshold currents, and substrate injection currents, which give rise to the static component of power dissipation. The largest percentage of static power results from source-to-drain subthreshold voltage, which is caused by reduced threshold voltages that prevent the gate from completely turning off. Leakage current was negligible in earlier CMOS technologies. However, with shrinking device geometries and reduced threshold voltages, leakage power is becoming increasingly significant, sometime approaching the levels of dynamic power dissipation.

The main causes of leakage power are reverse-bias p-n junction diode leakage, subthreshold leakage, and gate leakage. These leakage paths in a CMOS inverter are shown in figure 2.2.

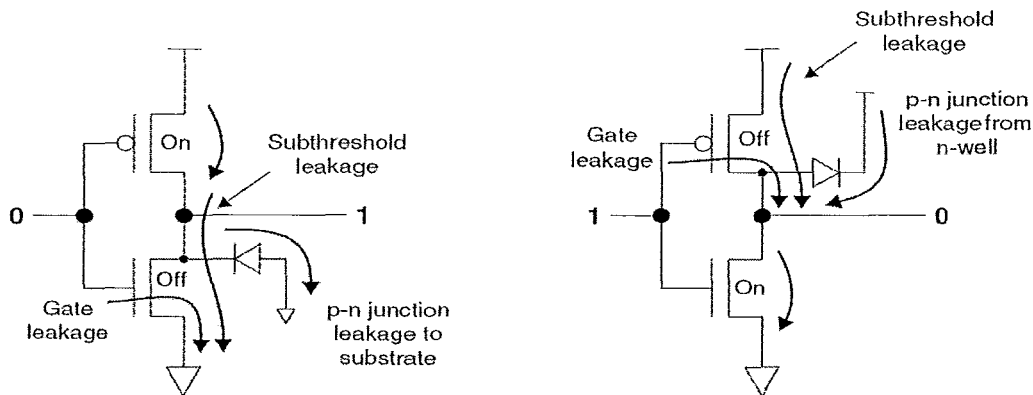


Figure: 2.2 - Static Leakage Currents

Leakage at reverse-biased p-n junctions (diode leakage) has always existed in CMOS circuits. This is the leakage from the n-type drain of the NMOS transistor to the grounded p-type substrate, and from the n-well (held at VDD) to the p-type drain of the PMOS transistor. This leakage is relatively small. Subthreshold leakage is the small source-to-drain current that flows even when the transistor is held in the “off” state. In older technologies, this current was negligible. However, with lower power supply voltages and lower threshold voltages, “off” gate voltages are getting close to “on” threshold voltages. Subthreshold leakage current increases exponentially as the gate voltage approaches the threshold voltage. Gate leakage is the result of using an extremely thin insulating layer between the gate conductor and the MOS transistor channel. Gate oxides are becoming so thin that only a dozen or fewer layers of insulating atoms separate the gate from the source and drain. Under these conditions, quantum-effect tunneling of electrons through the gate oxide can occur, resulting in significant leakage from the gate to the source or drain. Leakage currents occur whenever power is applied to the transistor, irrespective of the clock speed or switching activity. Slowing or stopping the clock cannot reduce leakage. However, it can be reduced or eliminated by lowering the supply voltage or by switching off the power to the transistors entirely.

2.2.2 Dynamic Power

Dynamic power is the power dissipated when the circuit is active. A circuit is active anytime the voltage on net changes due to some stimulus applied to the circuit. In other words, charging causes dynamic power dissipation. Because voltage on an input net can change without necessarily resulting in logic transition in the output, dynamic power can be dissipated even when an output net doesn't change its logic state. This component of dynamic power dissipation is the result of charging and discharging parasitic capacitances in the circuit.

Dynamic power of a circuit is composed of

- a) Switching power
- b) Internal power

2.2.2.1 Switching power

The switching power of a driving cell is the power dissipated by the charging and discharging of the load capacitance at the output of the cell. The total load capacitance at the output of a driving cell is the sum of the net and gate capacitances on the driving output. The charging and discharging are result of logic transitions. Switching power increases as logic transitions increase. Therefore, the switching power of a cell is a function of both the total load capacitance at the cell output and the rate of logic transitions. Switching power comprises 70-90 percent of the power dissipation of an active CMOS circuit. The cause of switching power is illustrated in Figure 2.3. A transition from 0 to 1 on the output of the inverter charges the capacitive load of the output net through the PMOS transistor. A transition from 1 to 0 discharges the same capacitive load through the NMOS transistor.

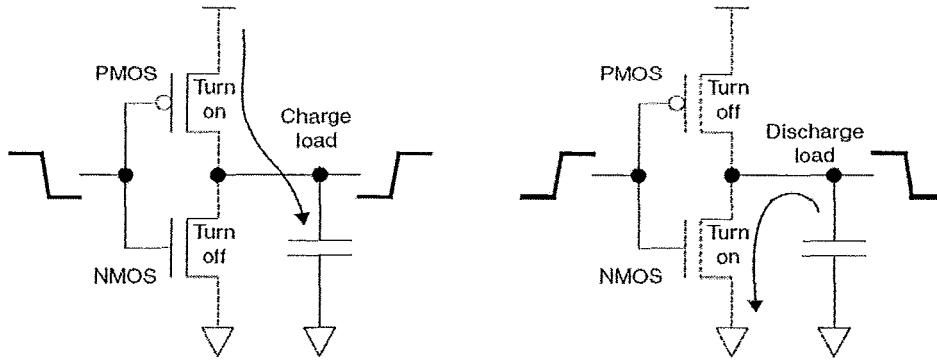


Figure: 2.3 - Switching Power

2.2.2.2 Internal power

Internal power is any power dissipated within the boundary of a cell. During switching, a circuit dissipates internal power by the charging or discharging of any existing capacitances internal to the cell. Internal power includes power dissipated by a momentary short circuit between the P and N transistors of a gate, called short-circuit power. In most simple library cells, internal power is due mostly to short-circuit power. Library developers can model internal power by using the internal power library group [2].

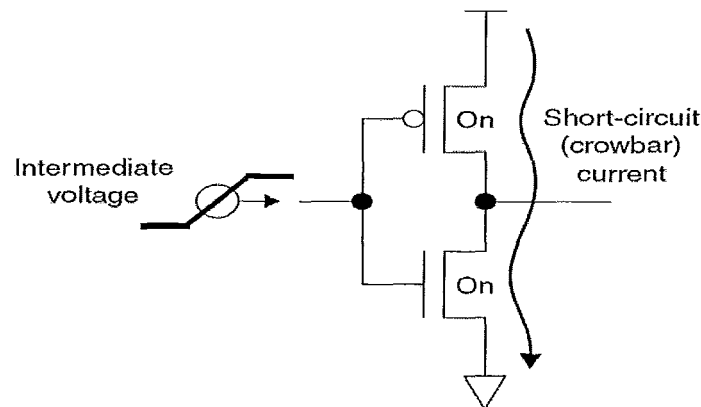


Figure: 2.4 - Internal Power

2.2.3 Short-Circuit Power

The short-circuit power consumption, $P_{\text{short-circuit}}$, is caused by the current flow through the direct path existing between the power supply and the ground during the transition phase.

2.2.4 Leakage Power

The NMOS and PMOS transistors used in a CMOS logic circuit commonly have non-zero reverse leakage and sub-threshold currents. These currents can contribute to the total power dissipation even when the transistors are not performing any switching action. The leakage power dissipation, P_{leakage} is caused by two types of leakage currents

- a) Reverse-bias diode leakage current
- b) Subthreshold current through a turned-off transistor channel [18].

2.2.5 Voltage drop effects

Deep submicron (DSM) and ultra-deep submicron (UDSM) devices are prone to voltage drop effects, which are caused by the resistance associated with the network of wires used to distribute power and ground from the external pins to the internal circuitry (in the case of DC related voltage drops, these are also often referred to as IR drop effects). Purely for the purposes of providing a simple example, consider a chain of inverter gates connected to the same power and ground tracks (Figure 2.5).

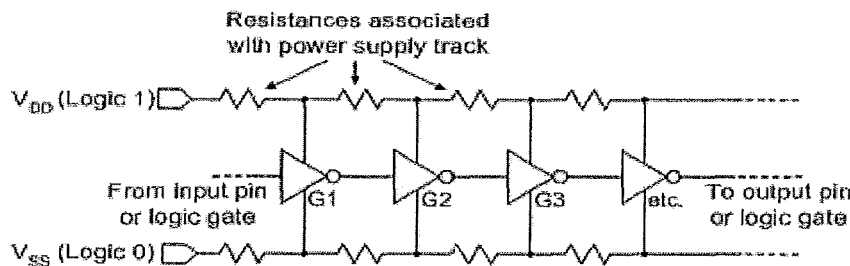


Figure: 2.5 - A chain of inverters connected to the same power and ground tracks [9]

Every power and ground track segment has a small amount of resistance associated with it. This means that the logic gate closest to the IC's primary power or ground pins (gate

G1 in this example) is presented with the optimal supply. The next gate in the chain (G2 in this example) will be presented with a slightly degraded supply, and so on down the chain. The problem is exacerbated in the case of transient or AC voltage drop effects. These occur when gates are switching from one value to another or even worse when entire blocks are switched on and off. This causes transitory power surges, which momentarily reduce the voltage supply to gates farther down the power supply chain.

The simple example circuit shown in Figure 2.5 consists only of inverter gates, but a real design typically contains tens of thousands of register (storage) elements triggered by a clock signal. The clock can cause large numbers of register elements to switch simultaneously, resulting in significant "glitches" in the power supply. In order to analyze and address these effects, it is necessary to take resistive, inductive, and capacitive effects into account. The reason voltage drop effects are so important is that the input-to-output delays across a logic gate increase as the voltage supplied to that gate is reduced, which can cause the gate to miss its timing specifications. There is also an increase in the interconnect delays associated with wires driven by underpowered gates. Furthermore, a gate's input switching thresholds are modified when its supply is reduced, which causes that gate to become more susceptible to noise.

Voltage drop effects are becoming increasingly significant, because the resistivity of the power and ground tracks rises as a function of decreasing feature sizes (track widths). These effects can be minimized by increasing the width of power and ground tracks, but this consumes valuable real estate on the silicon, which typically causes routing congestion problems.

In order to solve these problems, the logic functions have to be spaced farther apart, which increases delays (and power consumption) due to longer signal tracks. Thus, implementing an optimal power network requires the balancing of many diverse factors [18].

2.2.6 Electro migration effects

Electromigration occurs when the current density (current per cross-sectional area) in tracks is too high. In the case of power and ground tracks, electromigration effects are DC-based. The so-called "electron wind" induced by the current flowing through a track causes metal ions in the track to migrate. This migration creates "voids" in the "upwind" direction, while metal ions can accumulate "downwind" to form features called "hillocks" and "whiskers."

Electromigration in power and ground tracks causes timing problems, because the increased track resistance associated with a void can result in a corresponding voltage drop. This will, in turn, cause increased delays and noise susceptibility in affected logic gates as discussed above.

Power and ground electromigration can also cause major functional errors to occur, because the voids may eventually lead to open circuits while the hillocks and whiskers may cause short circuits to neighboring wires.

2.3 Low Power Design Techniques

Low power design is not a singular methodology but a host of techniques covering the entire range of design hierarchy from low (device silicon) to high (end-user operation), each having a different degree of impact on power saving. In general, the highest potential of energy reduction is realized at the higher end of design abstraction level [4]. The scope of this section is to introduce low power design techniques at the system and architectural level, where design engineers have the most influence in reducing device power consumption.

2.3.1. Voltage Scaling

A designer must manipulate equation 4 in order to maximize the power efficiency of a system. At a quick glance, it is clear that every term is dependent on the source voltage of the device. This would encourage designers to decrease the supply voltage of

the device, but this comes at a steep price. The delay time of a device is inversely proportional to its supply voltage, which is approximately [1]:

$$T_{delay} \propto \frac{C_{load} V_{DD}}{(V_{DD} - V_{threshold})^{1.5}} \quad (5)$$

As the supply voltage approaches the threshold voltage, the delay times become exceedingly larger. This tradeoff between power and speed is the energy-delay product. Examining equation 5 the energy-delay product is minimized when the source voltage is approximately twice as large as the threshold voltage. For a typical CMOS device, this will yield a 50% decrease in speed and a 44% decrease in power consumption [1]. Figure 2.62 is a plot of a CMOS device using typical values and a threshold voltage of one volt.

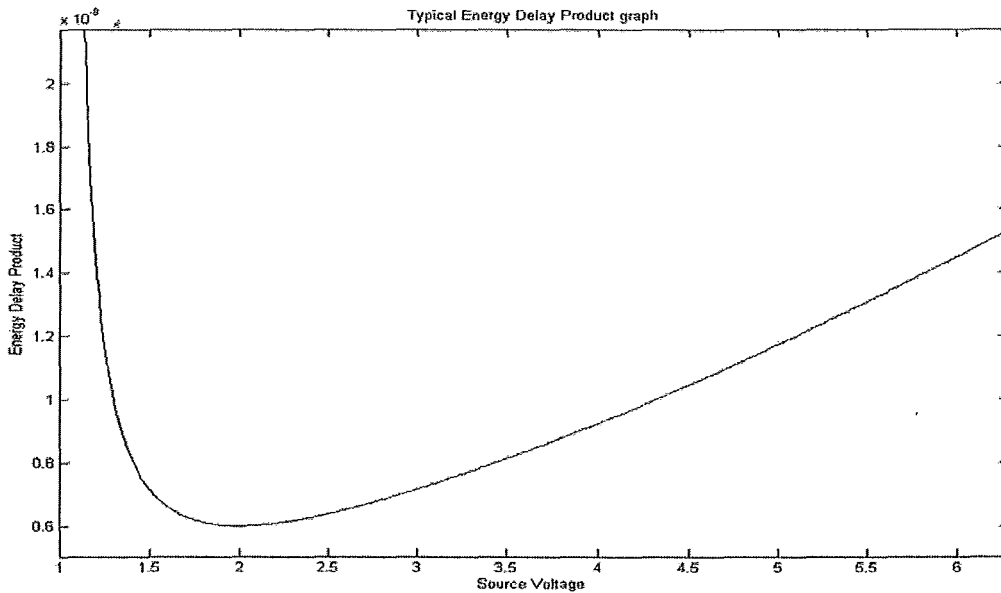


Figure: 2.6 - Energy product delay of a typical CMOS device

2.3.2. Parallelism and Pipelining

A major penalty from applying voltage scaling is the reduced speed the device will operate at. Architectural techniques such as parallelism and pipelining are useful in compensating for the degradation of data throughput which arises. In parallelism, the hardware architecture replicates a functional circuit multiple times allowing the

processing load to be spread across the circuits. Figure 2.7 shows an adder comparator circuit implemented using parallelism [5].

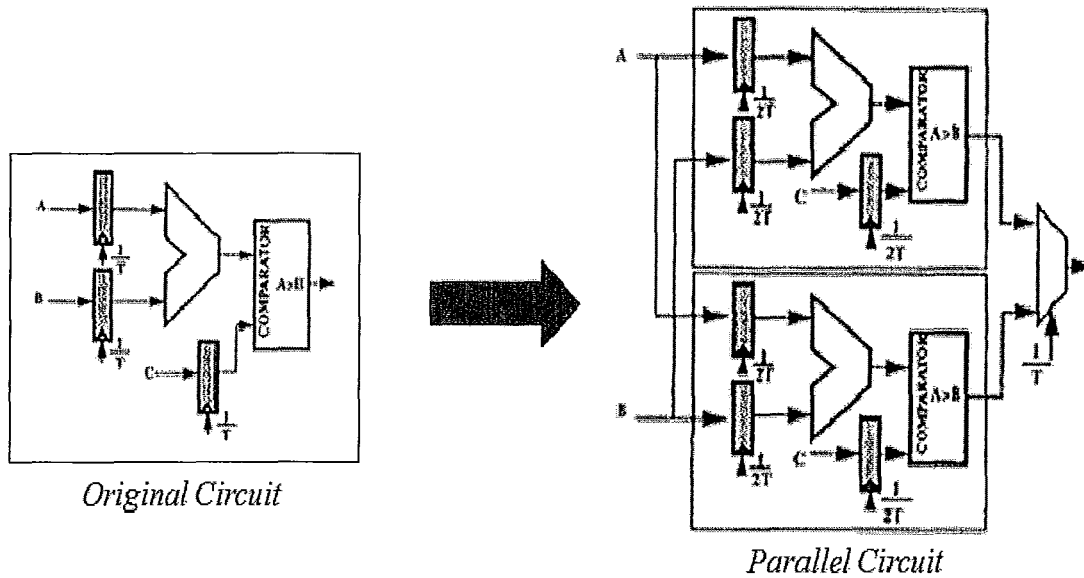


Figure: 2.7 - Parallel implementation of a circuit

The above example shows the two identical adder comparators will allow the processor to maintain the same data throughput while operating at half the clock frequency, designated in the figure as $1/2T$. The disadvantage of parallelism is also quite evident, the same function now requires more than twice the area with two identical circuit blocks and an extra overhead in the form of a multiplier. Unfortunately this extra overhead also increases power usage, so careful design considerations must be applied to avoid causing a detrimental effect on power consumption.

Aside from parallelism, a processor can also apply pipelining to maintain data throughput when the supply voltage is reduced. Pipelining reduces the critical path of a circuit by inserting additional registers in the data path, thus allowing the circuit to operate at the same clock frequency. Figure 2.8 shows the previously mentioned adder comparator circuit with pipelining [5].

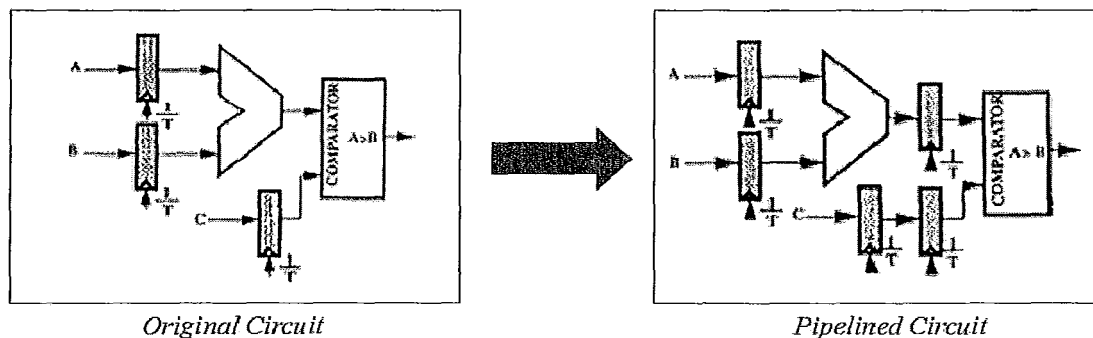


Figure: 2.8 - Pipelined implementation of a circuit

As can be seen from the figure 2.8, the pipelining registers will have a slight increase in area usage over the original design as well as increased the latency. As an added benefit, pipelining reduces logic depth and hence the power wasted because of potential hazards and critical races [5].

Parallelism and pipelining are not mutually exclusive techniques and can be combined to scale the supply voltage even lower, thus realizing even greater power efficiency. Table 1 compares voltage, area, and power requirement of these architectural techniques applied to a simple adder comparator circuit [5]. Notice that a parallel-pipeline implementation results in a five times reduction in power consumption.

Architecture Type	Voltage	Area	Power
No parallelism or pipelining	5 V	1	1
Parallel data path	2.9 V	3.4	0.36
Pipelined data path	2.9 V	1.3	0.39
Parallel and pipelined data path	2.0 V	3.7	0.2

Table: 1 - Architecture summary.

As can be seen, a simple time-multiplexed architecture is very inefficient for low power design. Using parallelism and pipelining allows the processor to reduce supply voltage without degrading data throughput performance. However, the overhead penalty associated with these techniques must be carefully weighed with the potential benefits based on the suitability to the individual circuits. A circuit that cannot take advantage of these techniques will incur increased area usage as well as power consumption with no appreciable boost in performance.

2.3.3. Device Enabling

It is important to note that two of the three types of power dissipations for a CMOS device occur only when the input and/or outputs are switched. Simply preventing the input of a subcomponent from changing will completely remove dynamic power dissipation in the component. This technique of temporarily disabling unused subcomponents is called device enabling. Disabling a component will reduce the overall dynamic power dissipation in the circuit; greatly reducing its power consumption.

Large components of a circuit can be designed with enable lines that allow an additional system controller to disable unused components. This will prevent a particular subsystem's output from switching by maintaining its input; temporarily removing the effects of $P_{\text{switching}}$, and $P_{\text{short-circuit}}$.

System controller scheduling is an example of a device enabling technique that is often implemented. In a simplified system, a design module will be continuously powered while spending a meager fraction of the time performing useful computation [6]. By taking into account the criteria of a process, such as priority, duration, and latency, the designer can optimize its power consumption. An energy aware scheduler will collect all requests for computation, then power up the module to do a continuous burst of computation. The design module will immediately power down as soon as all requests have been fulfilled, thus ensuring the system will operate at full efficiency when it is active [6]. Figure 2.9 shows a normal system operation compared with an energy aware scheduled operation. This method of intelligent scheduling incurs the overhead of

additional logic and memory to handle computation requests as well as a increased latency for the requested task.

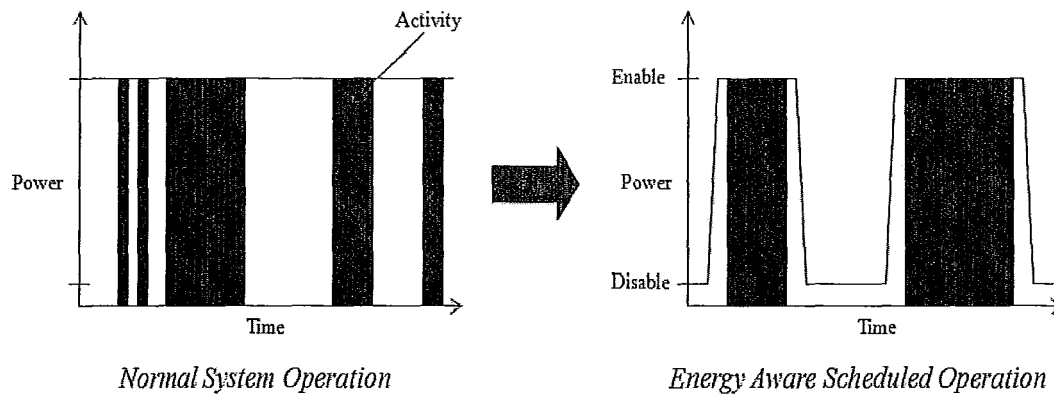


Figure: 2.9 - Scheduling

Clock gating is a slightly different technique that can accomplish similar results. Figure 2.10 is an example of using clock gating as a device enable. Clock gating can add an extra level of control by multiplexing different clock speed to various subcomponents [3]. For situations where a component cannot be disabled often, clock gating is an ideal way of slowing down components as well as disabling them. This will reduce the dynamic power dissipation as appropriate. However, this comes at the cost of a more complicated system controller. Gating a clock line also significantly increases the potential for glitches across one of the most vital signal in a design; great care must be taken to prevent hazards from propagating to the clock signal.

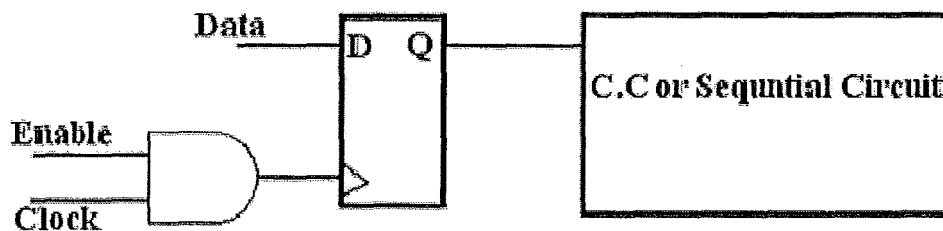


Figure: 2.10 - Clock gating example.

Device enabling is not always a practical solution to power efficiency. The process of device enabling requires extra hardware to implement which equates to larger space, added cost, as well as the power required to drive the system controller. The benefits of device enabling become apparent when used to disable large portions of a system as opposed to a few individual components.

2.3.4. Interconnect Minimization

The interconnection in a system contributes greatly to its power consumption. Experiments have demonstrated that in designs, about 10 to 40% of the total power may be dissipated in buses, multiplexers and drivers [6]. By implementing proper design partitioning techniques, this energy dissipation can be reduced.

The goal of design partitioning is essentially to maximize the use of short local buses over global busses. Global busses are longer with higher capacitance, requiring more frequent access to buffers, multiplexers and bus control logic as well as transistors with greater current drives. Figure 2.11 shows the simplicity of using local bus over global bus. However despite the minimal logic overhead, the preferential use of local buses also increases area since each local bus connection between modules is unique.

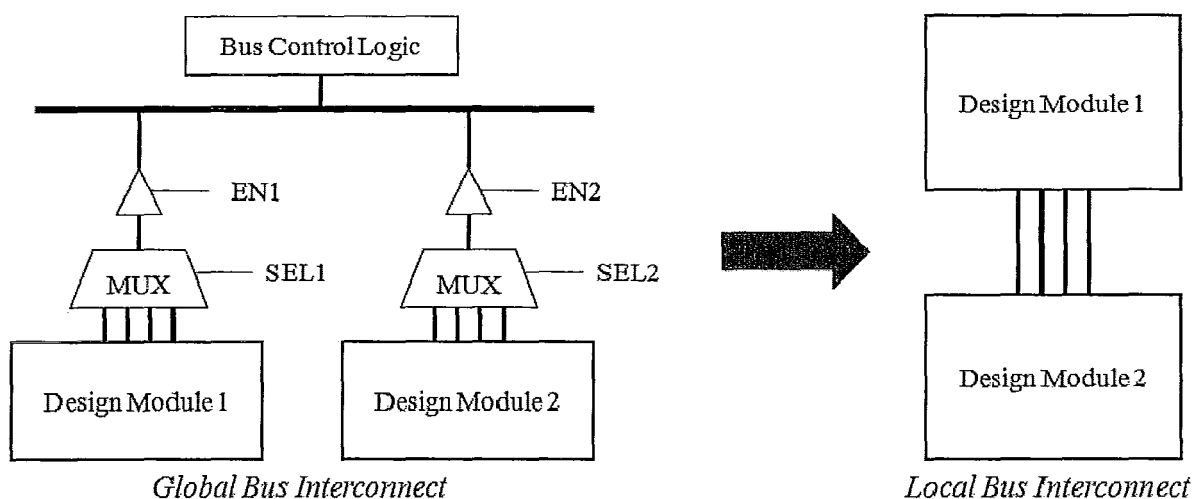


Figure: 2.11 - Module interconnection using global bus and local bus.

On a system level, placing design modules with frequent data transfers with each other closely using local buses for interconnection performs partitioning. This will ensure that each group of modules will have the majority of its data transfers within the group minimizing the use of system-wide global buses. Carefully partitioned designs will actually improve performance in addition to energy efficiency, due to the overhead associated with accessing the global bus.

Design partitioning can also be applied to memory design. Frequently accessed code or data can be placed in a small memory in the form of a cache near the calling module or processor. This has the benefit of reducing data traffic and power consumption on global memory buses as well as improving access time.

2.3.5. Logic Design Techniques

A number of techniques can be applied on the logic design level to improve power usage by a circuit. As a general rule of thumb the less logic a design uses, the more power efficient the design will be. The use of one hot encoding in finite state machine design will consume the least power since it has simpler state decoding logic than binary and Gray encoding. It also has the benefit of being typically faster. The type of counters selected may also play a role in power reduction. A Gray counter performs only one transition per clock cycle compared to either binary or LFSR counters in which all the bits of the counter may toggle on a given clock cycle. This unique property makes it the lowest power counter implementation [3]. However a Gray counter also requires more logic gates to implement so the complexity must be weighed with the power savings from single bit transitions.

Applying asynchronous design methodology will also help reduce power consumption since with the absence of a system clock, the gates are only active when they are switching. However as a tradeoff, asynchronous circuits are larger than synchronous designs and are more prone to hazards, both of which increase energy dissipation.

Wasteful activities on signal lines should also be optimized. A designer may accidentally create combinatorial loops in the logic design, which will oscillate indefinitely under certain inputs. Floating input signals will also consume excessive power and should be pulled high or low. Another example of wasteful activity is bus conflicts. Two drivers attempting to swing a data bus in opposite direction will severely affect power consumption, not to mention logic performance [7]. These wasteful gate transitions will create an excessive power draw and must be avoided.

In general, following good logic design guidelines will help reduce power consumption as well as create more stable designs. Designing more efficient circuits that use less logic gates and reduces gate transitions will reduce power. Guaranteeing logic design integrity will minimize hazards and race conditions eliminating unnecessary power usage as well as avoiding erroneous circuit operations [8].

CHAPTER: 3 - ASIC DESIGN FLOW AND IMPLEMENTATION

3.1 ASIC Design Flow

Application Specific Integrated Circuits(ASIC) are used to design entire systems on a single chip. ASICs are inter connect of standard cells which have been standardized by fabrication houses. With the integration of more and more system components on single IC, the complexity of IC fabrication has increased. Modern day system design involves complex layout issues. Specifications of cells are provided by the vendors in form of Technology library which contains information about geometry, delay and power characteristics of cells.[25]

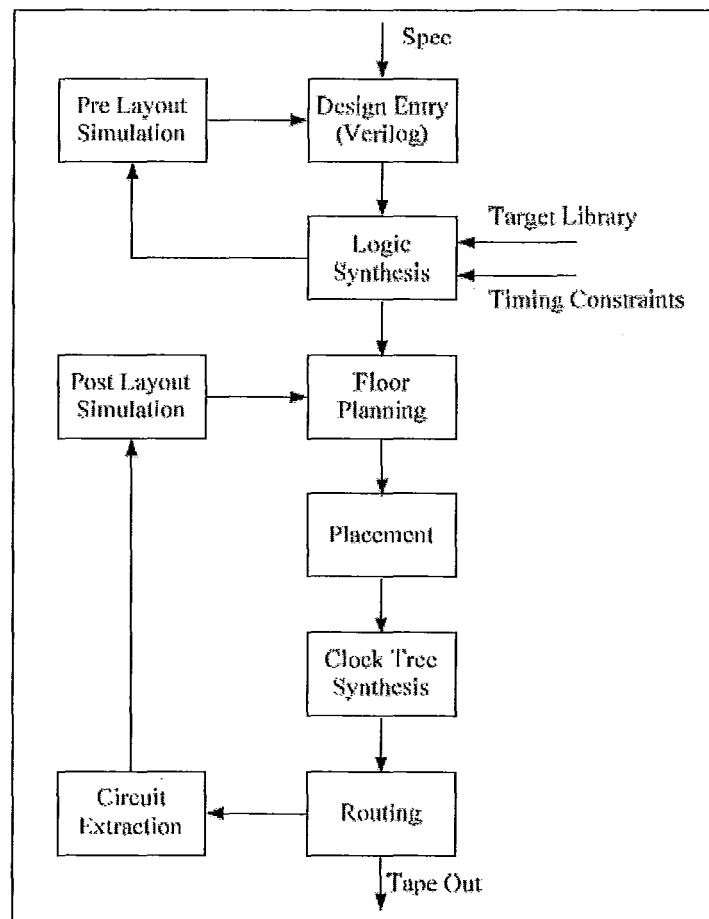


Figure: 3.1 - ASIC Design Flow

The steps of the ASIC flow are listed below with a brief description of the function of each step.

- Design entry: Enter the design into an ASIC design system using a hardware description language (HDL).
- Logic synthesis: Use an HDL (VHDL or Verilog) and a logic synthesis tool to produce a netlist - a description of the logic cells and their connections.
- Pre layout simulation: Check to see if the design functions correctly.
- Floor planning: Arrange the blocks of the netlist on the chip.
- Placement: Decide the locations of cells in a block.
- Clock tree synthesis: Insert and optimize clock tree and logic.
- Routing: Make the connections between cells and blocks.
- Circuit Extraction: Determine the resistance and capacitance of the inter-connect wire.
- Post layout simulation: Check to see the design still works with the added loads of the inter-connect wire [16].

3.2 ASIC Implementation

The implementation of a digital ASIC begins soon after the definition of the architecture. A Hardware Description Language (HDL), such as Verilog or VHDL is used to develop the Register Transfer Level (RTL) model of the overall system architecture. The RTL code emulates the process of transferring data between registers through combinational logic. Memory and other macro cells included on the chip are also modeled. The RTL model is validated by the application of test vectors, to ensure correct functionality. Test vectors refer to a sequence of inputs applied to the model, for which an expected response is known. By comparing the expected and actual response of the model, the validity of the RTL model can be determined and corrected as needed. The tool used to simulate this design is ModelSim, a HDL compiler and simulator. The compiled source code will be simulated using it.

After validating the RTL model, logic synthesis is performed to map the RTL code to logic gates in the targeted foundry library. Logic synthesis is the process of

Automatically converting a given RTL Hardware Descriptive Language of a design to Technology gates based on some design constraints. The result of the synthesis process is a Netlist composed of registers, combinational gates, interconnects, and macro cell instantiations. The tool used in this step is Design Compiler(DC) from Synopsys. DC is a gain based RTL synthesis tool that provides fast, high-capacity synthesis, integrated into an RTL-to-GDSII design flow.

The synthesized Netlist should be simulated using the same test vectors used to validate the RTL. In addition, Formal Verification software can be used to compare two given circuit descriptions. Formal Verification offers an alternative to exhaustive simulation. There is no need to create test vectors and thorough verification is achieved with in a short amount of time. The goal of formal verification is to ensure the equivalence of two given circuit descriptions. These circuits might be given on different levels of abstraction, i.e. register transfer level or gate level. Any simulation or formal verification errors need to be resolved since this may indicate a problem with the library of hardware components used by the synthesizer, a synthesis error, or improperly written RTL. Synthesis tools typically have little access to place and route data for the design. With little access to physical design data, synthesis tools can miss timing goals by as much as 100% based on the estimated versus the actual interconnect. Hence it is important that the synthesized design exceeds the final timing requirements. A good strategy is to use floor planning tools to perform preliminary placement of logic on a chip and to estimate interconnect delays based on that placement. This timing information can then be used to drive the front-end synthesis, simulation, and timing tools for more accurate results.

Once the design has been synthesized, the physical design process can begin. The first step in the physical design process is floor planning. The floorplan is a physical description of an ASIC. Floorplanning is a mapping between the logical description and the physical description of an ASIC. Floor planning involves partitioning the design into groupings of cells and placement of macro cells contained in the design on the chip area.

Other floor planning considerations are the creation of I/O macros and the power mesh structure. In order to meet the design objectives such as die-size and performance the

cells/structures that need to be placed close together are identified in the placement stage. The next step in the ASIC design flow is the Clock Tree Synthesis. Clock tree synthesis is performed in order to minimize space and power consumed by the clock signal. In this step the clock tree and clock logic is inserted and optimized. Once the clock tree is placed and optimized the place and route can begin. The interconnect wires are routed between the library components in the Netlist. After the place and route is completed the design is checked for DRC, LVS, Antenna violations and Crosstalk violations. Finally the parasitic data gathered from the place and routed Netlist is used to check if the design meets timing. The tool used to layout the design is Synopsys's IC compiler. IC compiler is a physical design system that provides a complete Netlist-to-GDSII flow for cell-based ICs. IC compiler includes optimization, place and route, useful skew clock generation; Floorplanning and power planning; RC extraction; and a single, built-in Incremental timing analyzer

CHAPTER: 4 - Design Compiler

The Design Compiler tool is the core of the Synopsys synthesis products. Design Compiler optimizes designs to provide the smallest and fastest logical representation of a given function. It comprises tools that synthesize your HDL designs into optimized technology-dependent, gate-level designs. It supports a wide range of flat and hierarchical design styles and can optimize both combinational and sequential designs for speed, area, and power.

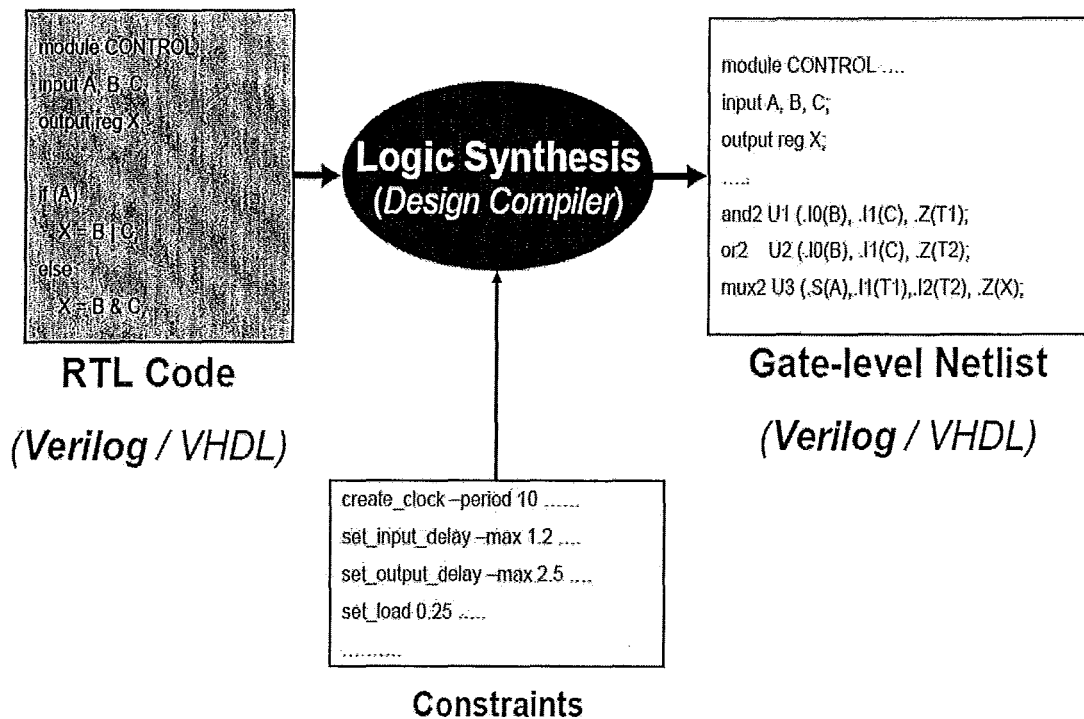


Figure: 4.1 - Design Compiler

The following figure shows the inputs to Design Compiler which is an RTL code and Timing constraints. After synthesis it gives a Gate-level Netlist as output, this netlist could be obtained as .ddc file format (Mapped Netlist).

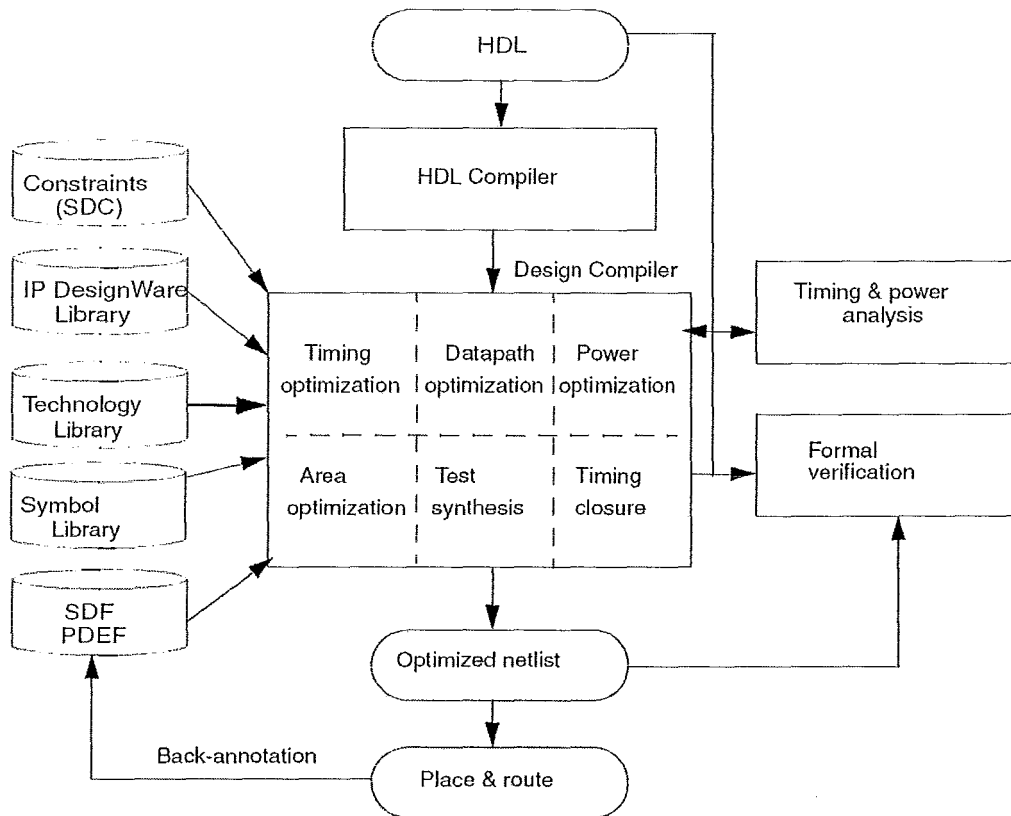


Figure: 4.2 - Design Compiler Flow

Design Compiler is used for logic synthesis, which is the process of converting a design description written in a hardware description language such as verilog or VHDL into an optimized gate-level netlist mapped to a specific technology library. The steps in the synthesis process are as follows:

1. The input design files for Design Compiler are often written using a hardware description language (HDL) such as verilog or VHDL.
2. Design Compiler uses technology libraries, synthetic or DesignWare libraries, and symbol libraries to implement synthesis and to display synthesis results graphically. During the synthesis process, Design Compiler translates the HDL description to components extracted from the generic technology (GTECH) library and DesignWare library. The GTECH library consists of basic logic gates and flip-flops. The DesignWare library contains more complex cells such as adders and comparators. Both the GTECH and DesignWare libraries are

technology independent, that is, they are not mapped to a specific technology library. Design Compiler uses the symbol library to generate the design schematic.

3. After translating the HDL description to gates, Design Compiler optimizes and maps the design to a specific technology library, known as the target library. The process is constraint driven. Constraints are the designer's specification of timing and environmental restrictions under which synthesis is to be performed.
4. After the design is optimized, it is ready for test synthesis. Test synthesis is the process by which designers can integrate test logic into a design during logic synthesis. Test synthesis enables designers to ensure that a design is testable and resolve any test issues early in the design cycle. The result of the logic synthesis process is an optimized gate-level netlist, which is a list of circuit elements and their interconnections.
5. After test synthesis, the design is ready for the place and route tools, which place and interconnect cells in the design. Based on the physical routing, the designer can back-annotate the design with actual interconnect delays; Design Compiler can then re-synthesize the design for more accurate timing analysis.

HDL coding is the foundation for synthesis because it implies the initial structure of the design. When writing a HDL source code, always consider the hardware implications of the code. A good coding style can generate smaller and faster designs. An efficient code will help in achieving the design target in the shortest possible time.

4.1 What is Synthesis?

Synthesis is the process that generates a gate-level netlist for an IC design that has been defined using a Hardware Description Language (HDL). Synthesis includes reading the HDL source code and optimizing the design from that description. Figure 4.3 gives a clear picture of how the RTL code is synthesized using Design Compiler.

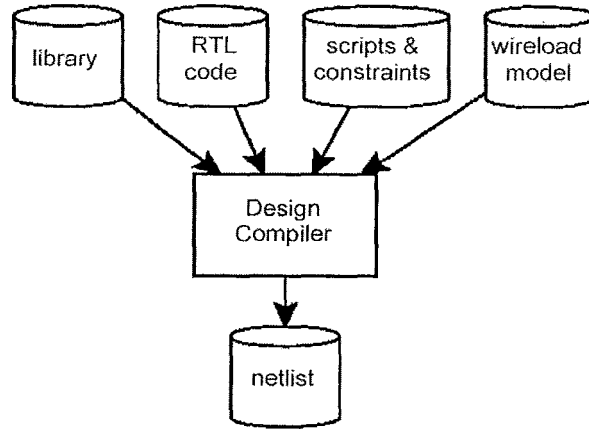


Figure: 4.3 - Synthesis in DC

4.2 System Library Variables (Linked & Target Library)

The `target_library` specifies the name of the technology library that corresponds to the library whose cells the designers want DC to infer and finally map to. The `link_library` defines the name of the library that refers to the library of cells used solely for reference, i.e., cells in the `link_library` are not inferred by DC.

4.2.1 Variables

Variables are placeholders used by DC for the purpose of storing information. The information may relate to instructions for tailoring the final netlist, or it may contain user-defined value to be used for automating the synthesis process. Some variables are pre-defined by DC and may be used by the designer to obtain the current value stored in the variable. All variables are global and last only during the session. They are not saved along with the design database.

4.2.2 Attributes

Attributes are similar in nature to variables. Both store information. However, attributes store information on a particular design object such as nets, cells or clocks. Generally, attributes are pre-defined and have special meaning to DC, though designers may set their own attributes if desired. For example, the `set_dont_touch` is a pre-defined attribute, used to set a `dont_touch` on a design, thereby disabling DC optimization on that design.

4.3 Specifying Libraries

Design Compiler uses these libraries:

- Technology libraries:
- Symbol libraries
- Design Ware libraries

4.3.1 Technology Libraries

Technology libraries contain information about the characteristics and functions of each cell provided in a semiconductor vendor's library. Semiconductor vendors maintain and distribute the technology libraries.

The Synopsys technology libraries can be separated in two broad classes:

1. Logic library
2. Physical library

4.3.2 Logic Library

The logic library contains information relevant only to the synthesis process and is used by DC for synthesis and optimization of the design. This information may include pin-to-pin timing, area, pin types and power along with other necessary data needed by DC. No physical information is present in the logic library. The logic library is a text file (usually with extension “.lib”), which is compiled using the Library Compiler (LC) to generate a binary format with “.db” extension.

The logic library contains the following information:

1. Library group
2. Library level attributes
3. Environment description
4. Cell description

4.3.3 Physical Library

The physical library contains the physical characteristics of the cell along with other necessary information relevant to Physical Compiler. Such information may contain data relating to the physical dimensions of cells, layer information, orientation of cells etc. For each logical cell, a corresponding physical cell should also be present.

4.3.4 Symbol Libraries

Symbol libraries contain definitions of the graphic symbols that represent library cells in the design schematics. Semiconductor vendors maintain and distribute the symbol libraries. Design Compiler uses symbol libraries to generate the design schematic. The `symbol_library` system variable holds the name of the library, containing graphical representation of the cells in the technology library. It is used, to represent the gates schematically, while using the graphical frontend tool. The symbol libraries are identified with “sdb” extension. Generally, all technology libraries provided by the library vendor include a corresponding symbol library. It is imperative that there be an exact match of the cell names and the pin names, between the technology and the symbol library. Any mismatch in a cell will cause the tool to reject the cell from the symbol library, and use the cell from the generic library.

4.3.5 Design Ware Libraries

A Design Ware library is a collection of reusable circuit-design building blocks (components) that are tightly integrated into the Synopsys synthesis environment. Design Ware components that implement many of the built-in HDL operators are provided by Synopsys. These operators include +, -, *, <, >, <=, >=, and the operations defined by if and case statements.

4.4 Design Flow

4.4.1 Read Design

Design Compiler reads designs into memory from design files. Many designs can be in memory at any time. After a design is read in, you can change it in numerous ways, such

as grouping or ungrouping its sub designs or changing sub design references. Design Compiler provides the following ways to read design files:

- The analyze and elaborate commands
- The read_file command

Using the analyze and elaborate Commands

The analyze command does the following:

- Reads an HDL source file
- Checks it for errors (without building generic logic for the design)
- Creates HDL library objects in an HDL-independent intermediate format
- Stores the intermediate files in a location you define

If the analyze command reports errors, fix them in the HDL source file and run analyze again. After a design is analyzed, you must reanalyze it only when you change it.

The *elaborate* command does the following:

- Translates the design into a technology-independent design (GTECH) from the intermediate files produced during analysis
- Allows changing of parameter values defined in the source code
- Allows VHDL architecture selection
- Replaces the HDL arithmetic operators in the code with DesignWare components
- Automatically executes the link command, which resolves design references

Resolving the reference means that the design library or file containing the detailed design data for the sub-block can be found and processed. If any references in the netlist cannot be resolved, the link command will issue warnings as to which sub-component designs are not available.

4.4.2 Define Design Environment

In order to obtain optimum results from DC, designers have to methodically constrain their designs by describing the design environment, target objectives and design rules. The constraints may contain timing and/or area information, usually derived from design specifications. DC uses these constraints to perform synthesis and tries to optimize the design with the aim of meeting target objectives. You define the environment by

specifying operating conditions, wire load models, and system interface characteristics. Operating conditions include temperature, voltage, and process variations. Wire load models estimate the effect of wire length on design performance. System interface characteristics include input drives, input and output loads, and fan-out loads. The environment model directly affects design synthesis results [11].

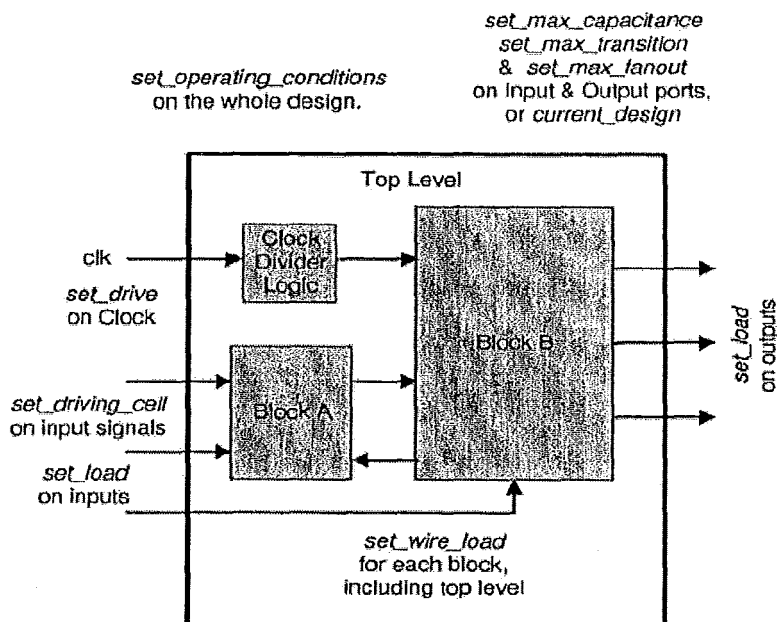


Figure: 4.4 - Basic Design Environment

- **set_operating_conditions** describes the process, voltage and temperature conditions of the design. The Synopsys library contains the description of these conditions, usually described as WORST, TYPICAL and BEST case. The names of operating conditions are library dependent. Users should check with their library vendor for correct setting. By changing the value of the operating condition command, full ranges of process variations are covered. The WORST case operating condition is generally used during pre-layout synthesis phase, thereby optimizing the design for maximum setup-time. The BEST case condition is commonly used to fix the hold-time violations. The TYPICAL case is mostly ignored, since analysis at WORST and BEST case also covers the TYPICAL case.

`set_operating_conditions <name of operating conditions>`

It is possible to optimize the design both with the WORST and the BEST case, simultaneously. The optimization is achieved by using the `-min` and `-max` options in the

above command, as illustrated below. This is very useful for fixing the design for possible hold-time violations. [11]

```
dc_shell> set_operating_conditions -min BEST -max WORST
```

- **set_wire_load_model** command is used to provide estimated statistical wire-load information to DC, which in turn, uses the wire-load information to model net delays as a function of loading. Generally, a number of wire-load models are present in the Synopsys technology library, each representing a particular size block. In addition, designers may also choose to create their own custom wire-load models to accurately model the net loading of their blocks.

```
set_wire_load_model -name <wire-load model>
```

- **set_drive** and **set_driving_cell** are used at the input ports of the block. **set_drive** command is used to specify the drive strength at the input port. It is typically used to model the external drive resistance to the ports of the block or chip. The value of 0 signifies highest drive strength and is commonly utilized for clock ports. Conversely, **set_driving_cell** is used to model the drive resistance of the driving cell to the input ports. This command takes the name of the driving cell as its argument and applies all design rule constraints of the driving cell to the input ports of the block.

```
set_drive <value> <object list>
```

```
set_driving_cell -cell <cell name>
```

```
-pin <pin name> <object list>
```

- Design Rule Constraints or DRCs consist of **set_max_transition**, **set_max_fanout** and **set_max_capacitance** commands. These rules are generally set in the technology library and are determined by the process parameters. These rules should not be violated in order to achieve working silicon. The DRC commands can be applied to input ports, output ports or on the `current_design`. Furthermore, if the value set in the technology library is not adequate or is too optimistic, then these commands may also be used at the command line, to control the buffering in the design.

```
set_max_transition <value> <object list>
```

```
set_max_capacitance <value> <object list>
```

```
set_max_fanout <value> <object list>
```

4.4.3 Design Constraints

Design constraints describe the goals for the design. They may consist of timing or area constraints. Depending on how the design is constrained, DC tries to meet the set objectives. It is imperative that designers specify realistic constraints, since unrealistic specification results in excess area, increased power and/or degradation in timing. The basic commands to constrain a design are shown in Figure 4.5 [11].

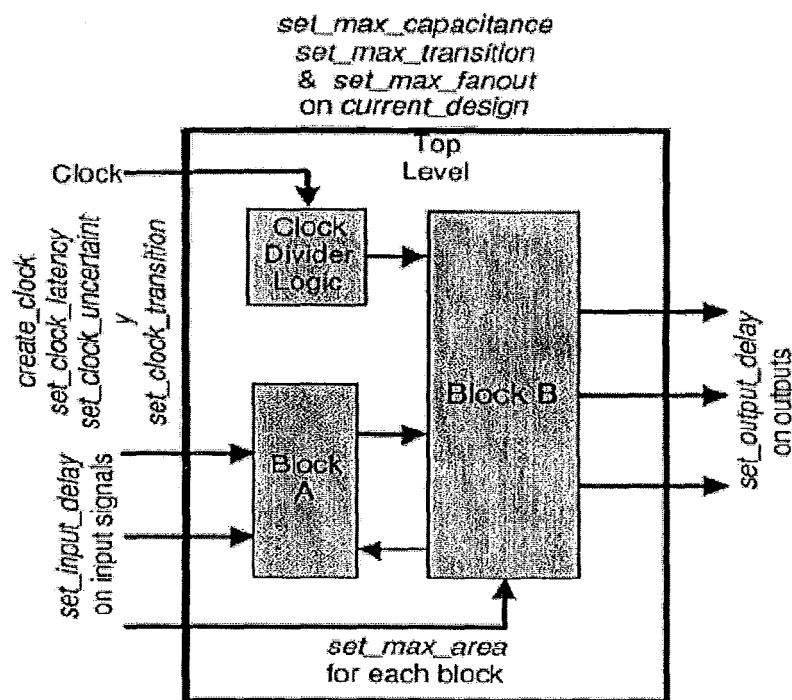


Figure: 4.5 - Design Constraints for synthesis

There are two categories of design constraints:

- Design rule constraints
- Design optimization constraints

Design rule constraints are supplied in the technology library we specify. They are referred to as the implicit design rules. These rules are established by the library vendor,

and, for the proper functioning of the fabricated circuit, they must not be violated. We can, however, specify stricter design rules if appropriate. The rules you specify are referred to as the explicit design rules.

Design optimization constraints define timing and area optimization goals for Design Compiler. These constraints are user-specified. Design Compiler optimizes the synthesis of the design, in accordance with these constraints, but not at the expense of the design rule constraints. That is, Design Compiler attempts never to violate the higher-priority design rules. [1]

- **create_clock** command is used to define a clock object with a particular period and waveform. The `-period` option defines the clock period, while the `-waveform` option controls the duty cycle and the starting edge of the clock. This command is applied to a pin or port, object types.

In some cases, a block may only contain combinational logic. To define delay constraints for this block, one can create a virtual clock and specify the input and output delays in relation to the virtual clock. To create a virtual clock, designers may replace the port name (CLK, in the above example) with the `-name <virtual clock name>`, in the above command. Alternatively, one can use the `set_max_delay` or `set_min_delay` commands to constrain such blocks.

-**create_generated_clock** command is used for clocks that are generated internal to the design. This command may be used to describe frequency divided/multiplied clocks as a function of the primary clock.

```
create_generated_clock -name <clock name>
                        -source <clock source>
                        -divide_by <factor> | -multiply_by <factor>
                        .....
```

- **set_dont_touch** is used to set a `dont_touch` property on the `current_design`, cells, references or nets. This command is frequently used during hierarchical compilation of

the blocks. Also, it can be used for, preventing DC from inferring certain types of cells present in the technology library.

-set_input_delay specifies the input arrival time of a signal in relation to the clock. It is used at the input ports to specify the time it takes for the data to be stable after the clock edge. The timing specification of the design usually contains this information, as the setup/hold time requirements for input signals. Given the top-level timing specification of the design, this information may also be extracted for the sub-blocks of the design.

In Figure 4.6, the maximum input delay constraint of 23ns and the minimum input delay constraint of 0ns is specified for the signal *datain* with respect to the clock signal *CLK*, with a 50% duty cycle and a period of 30ns. In other words the setup-time requirement for the input signal *datain* is 7ns, while the hold-time requirement is 0ns.

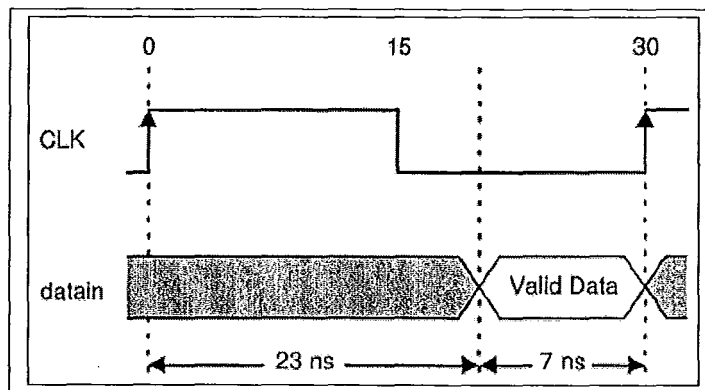


Figure: 4.6 - Specification of the Input Delay

- set_output_delay command is used at the output port to define the time it takes for the data to be available before the clock edge. The timing specification of the design usually contains this information. Given the top-level timing specification of the design, this information may also be extracted for the sub-blocks of the design.

In Figure 4.7, the output delay constraint of 19ns is specified for the signal *dataout* with respect to the clock signal *CLK*, with a 50% duty cycle and a period of 30ns. This means that the data is valid for 11ns after the clock edge.

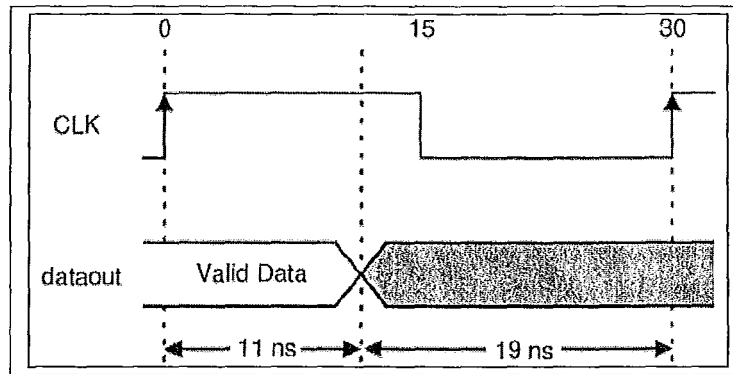


Figure: 4.7 - Specification of the Output Delay

- **set_clock_latency** command is used to define the estimated clock insertion delay during synthesis. This is primarily used during the prelayout synthesis and timing analysis. The estimated delay number is an approximation of the delay produced by the clock tree network insertion (done during the layout phase).

- **set_clock_uncertainty** command lets the user define the clock skew information. Basically this is used to add a certain amount of margin to the clock, both for setup and hold times. During the pre-layout phase one can add more margin as compared to the post-layout phase.

- **set_false_path** is used to instruct ICC to ignore a particular path for timing or optimization. Identification of false paths in a design is critical. Failure to do so compels DC to optimize all paths in order to reduce total negative slack. Consequently, the critical timing paths may be adversely affected due to optimization of all the paths, which also includes the false paths. The valid start point and endpoint to be used for this command are the input ports or the clock pins of the sequential elements, and the output ports or the data pins of the sequential cells.

- **set_max_delay** defines the maximum delay required in terms of time units for a particular path. In general, it is used for the blocks that contain combinational logic only. However, it may also be used to constrain a block that is driven by multiple clocks, each with a different frequency.

- **set_min_delay** is the opposite of the **set_max_delay** command, and is used to define the minimum delay required in terms of time units for a particular path.

4.4.4 Optimization Process

Optimization is the IC Compiler synthesis step that maps the design to an optimal combination of specific target library cells, based on the design's functional, speed, and area requirements. IC Compiler provides options that enable us to customize and control optimization. IC Compiler performs the following three levels of optimization:

- Architectural optimization
- Logic-level optimization
- Gate-level optimization

Architectural Optimization

Architectural optimization works on the HDL description. It includes such high-level synthesis tasks as

- Sharing common subexpressions
- Sharing resources
- Selecting DesignWare implementations
- Reordering operators

Logic-Level Optimization

Logic-level optimization works on the GTECH netlist. It consists of following two processes:

- Structuring
- Flattening

Gate-Level Optimization

Gate-level optimization works on the generic netlist created by logic synthesis to produce a technology-specific netlist. It includes the following processes:

- Mapping
- Delay optimization
- Design rule fixing
- Area optimization

4.4.5 Analyzing and Resolving Design Problems

The reports generated by Design Compiler are used to analyze and debug the design. We can generate reports both before and after you compile your design. Generate reports before compiling to check that you have set attributes, constraints, and design rules properly. Generate reports after compiling to analyze the results and debug the design.

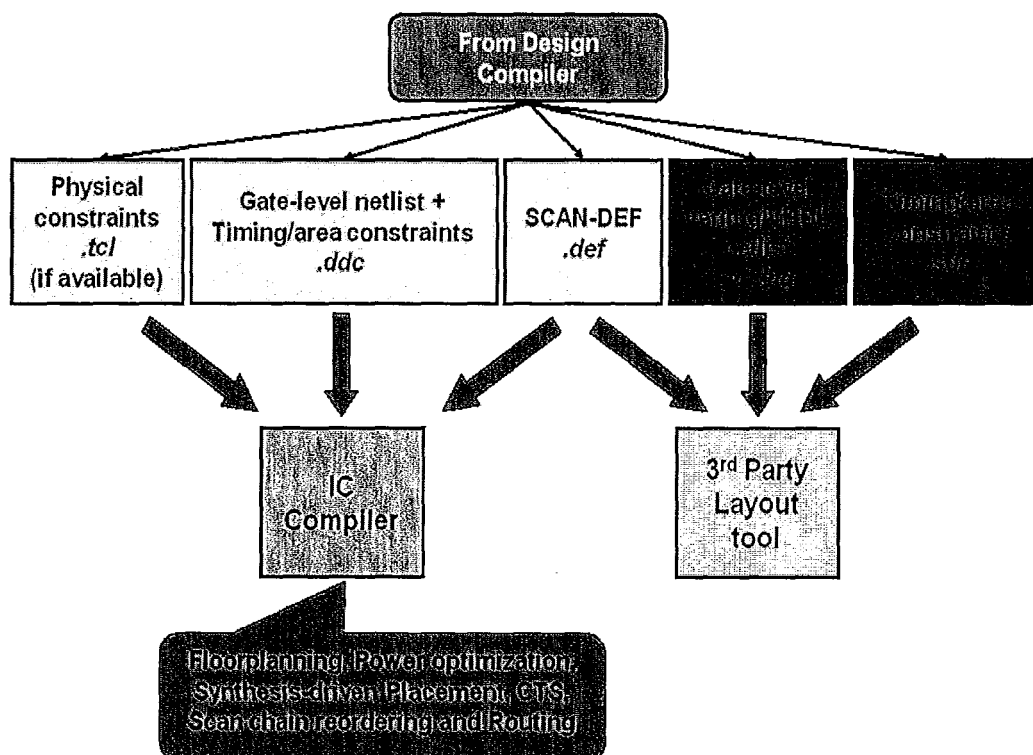


Figure: 4.8 - DC to ICC

CHAPTER: 5 - IC Compiler

IC Compiler is an integral part of the Synopsys Galaxy™ Design Platform that delivers a complete design solution, including synthesis, physical implementation, low-power design, and design for manufacturability (DFM). IC Compiler is a single, convergent, chip-level physical implementation tool that includes flat and hierarchical design planning, placement and optimization, clock tree synthesis, routing, DFM, and low-power capabilities that enable designers to implement today's high-performance, complex designs. Widely adopted and recognized as the industry standard for physical implementation.

IC Compiler advances ease-of-use with core commands (`place_opt`, `clock_opt`, and `route_opt`) to deliver best results. The IC Compiler GUI provides intuitive and easy-to-use features that enable designers to resolve issues at all design stages. The GUI enables fast analysis, visualization, debugging and repair features.

All of these shared technologies and key advances in IC Compiler enable the Galaxy Design Platform to deliver the best QoR in terms of timing, area, power, routability, testability and yield as well as faster TAT and a predictable path to first-silicon success. Today designers are using IC Compiler successfully to tape out numerous complex, high-performance, and low-power designs at 130nm to 45nm and below geometries.

Features:

- High throughput for designs in
- mainstream silicon technologies
- High performance for advanced silicon technologies
- Comprehensive optimization capabilities meet timing, area, power, signal integrity, routability and yield objectives
- Predictability during the implementation process
- Single timer
- Complete netlist-to-GDSII solution for best QoR and TTR
- Sign-off

- Highly correlated with golden sign-off solutions: PrimeTime SI and Star-RCXT
- Shares common infrastructure and technologies with PrimeTime, such as Arnoldi, OCV, CRPR, CCS, common cell delay calculation and SDC constraints to ensure tight correlation
- Improves TTR by eliminating unnecessary margins
- Speeds design closure by using exact sign-off timing and extraction information

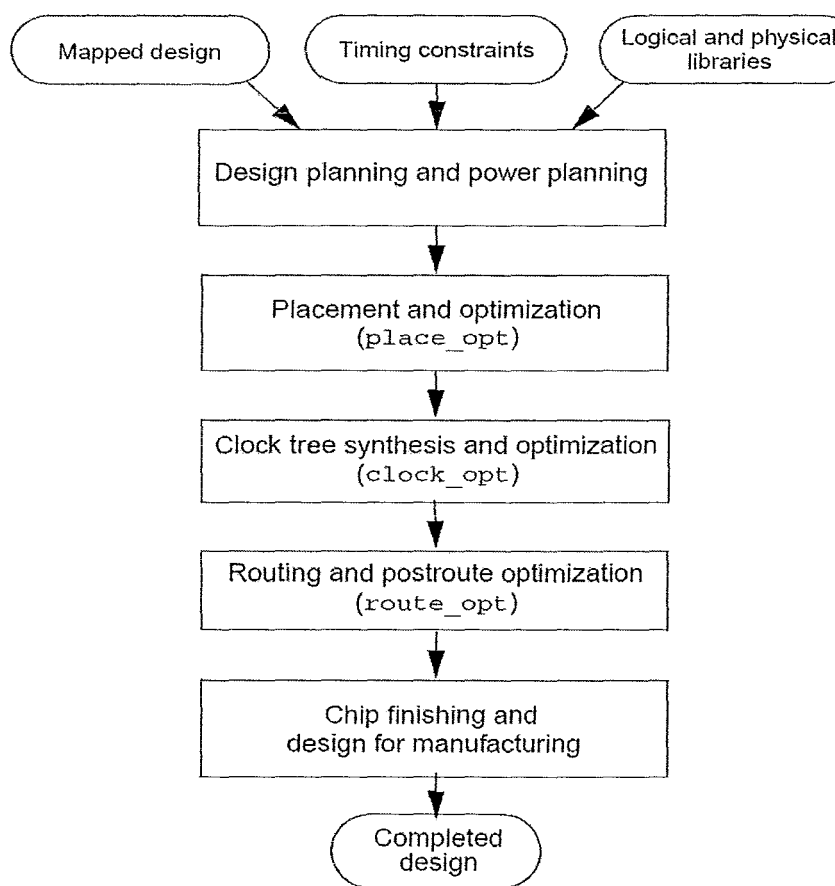


Figure: 5.1 - IC Compiler Design Flow

The IC Compiler design flow is an easy-to-use, single-pass flow that provides convergent timing closure. Figure 5.1 shows the basic IC Compiler design flow, which is centered on three core commands that perform placement and optimization (`place_opt`), clock tree synthesis and optimization (`clock_opt`), and routing and postroute optimization (`route_opt`). For most designs, the `place_opt`, `clock_opt`, and `route_opt` steps are preset for

optimal results. IC Compiler also provides additional placement, clock tree synthesis, and routing technologies, as well as extended physical synthesis tools, that you can use to further improve the quality of results for your design.

To run the IC Compiler design flow,

1. Set up the libraries and prepare the design data.

Perform design planning and power planning. When you perform design planning and power planning, you create a floorplan to determine the size of the design, create the boundary and core area, create site rows for the placement of standard cells, set up the I/O pads, and create a power plan.

2. Perform placement and optimization.

To perform placement and optimization, use the `place_opt` core command. IC Compiler placement and optimization addresses and resolves timing closure for your design. This iterative process uses enhanced placement and synthesis technologies to generate a legalized placement for leaf cells and an optimized design. You can supplement this functionality by optimizing for power, recovering area for placement, minimizing congestion, and minimizing timing and design rule violations. The Direct Timing- Driven Placement engine (DTDP) is used in the mainstream `place_opt` flow.

3. Perform clock tree synthesis and optimization.

To perform clock tree synthesis and optimization, use the `clock_opt` core command, IC Compiler clock tree synthesis and embedded optimization solve complicated clock tree synthesis problems, such as blockage avoidance and the correlation between preroute and postroute data. Clock tree optimization improves both clock skew and clock insertion delay by performing buffer sizing, buffer relocation, gate sizing, gate relocation, level adjustment, reconfiguration, delay insertion, dummy load insertion, and balancing of interclock delays.

4. Perform routing and postroute optimization.

To perform routing and postroute optimization, use the `route_opt` core command. As part of routing and postroute optimization, IC Compiler performs global routing, track assignment, detail routing, search and repair, topological optimization, and engineering change order (ECO) routing. For most designs, the default routing and

postroute optimization setup produces optimal results. If necessary, you can supplement this functionality by optimizing routing patterns and reducing crosstalk or by customizing the routing and postroute optimization functions for special needs.

5. Perform chip finishing and design for manufacturing tasks, IC Compiler provides chip finishing and design for manufacturing and yield capabilities that you can apply throughout the various stages of the design flow to address process design issues encountered during chip manufacturing.

5.1 Design Objects

There are eight different types of design objects categorized by ICC. These are:

Design: It corresponds to the circuit description that performs some logical function. The design may be stand-alone or may include other sub-designs. Although, sub-designs may be part of the design, it is treated as another design by Synopsys.

Cell: It is the instantiated name of the sub-design in the design. In Synopsys terminology, there is no differentiation between the cell and instance; both are treated as cell.

Reference: This is the definition of the original design to which the cell or instance refers. For example, a leaf cell in the netlist must be referenced from the link library, which contains the functional description of the cell. Similarly, a sub-design instantiated (called cell by Synopsys) must be referenced in the design, which contains functional description of the instantiated sub-design.

Port: These are the primary inputs, outputs or IO's of the design.

Pin: It corresponds to the inputs, outputs or IO's of the cells in the design

Net: These are the signal names, i.e., the wires that hook up the design together by connecting ports to pins and/or pins to each other.

Clock: The port or pin that is identified as a clock source. The identification may be internal to the library

Library: Corresponds to the collection of technology specific cells that the design is targeting for synthesis; or linking for reference.

5.2 Preparing the Design

IC Compiler uses a Milkyway (MW) design library to store your design and its associated library information. Creating a design library results in the creation of a UNIX directory with the same name, inside your present working directory. This library data structure or design database is called Milkyway. This chapter describes how to set up the libraries, create a Milkyway design library, read your design, and save the design in Milkyway format. These steps are explained in the following sections:

- Setting Up the Libraries
- Setting Up the Power and Ground Nets
- Reading the Design
- Annotating the Physical Data
- Preparing for Timing Analysis and RC Calculation
- Saving the Design

5.3 Setting up the Libraries

IC Compiler requires both logic libraries and physical libraries.

5.3.1 Setting up the Logic Libraries

IC Compiler uses logic libraries to provide timing and functionality information for all standard cells. In addition, logic libraries can provide timing information for hard macros, such as RAMs. IC Compiler supports logic libraries that use nonlinear delay models (NLDMs), Composite Current Source (CCS) models (either compact or non compact), or both. IC Compiler automatically selects the timing model to use based on the contents of the logic libraries. If the logic libraries contain only CCS models, IC Compiler uses CCS models. If the libraries contain nonlinear delay models (NLDMs) or both NLDM and Composite Current Source (CCS) models, IC Compiler uses the NLDM models. The selected models are used for both pre route and post route delay calculations.

IC Compiler uses variables to define the logic library settings. In each session, you must define the values for the following variables (either interactively, in the

synopsys_dc.setup file, or by restoring the values saved in the Milkyway design library) so that IC Compiler can access the libraries:

- **search_path**

Lists the paths where IC Compiler can locate the logic libraries.

- **target_library**

Lists the logic libraries that IC Compiler can use to perform physical optimization

- **link_library**

Lists the logic libraries that IC Compiler can search to resolve references

5.3.2 Setting up the Physical Libraries

IC Compiler uses Milkyway reference libraries and technology (.tf) files to provide physical library information. The Milkyway reference libraries contain physical information about the standard cells and macro cells in your technology library. In addition, these reference libraries define the placement unit tile. The technology files provide technology-specific information such as the names and characteristics (physical and electrical) for each metal layer.

The technology file contains the following details:

- Layer and via Definitions
- Process design rules (Minimum metal widths and spacing)
- Resistance / Capacitance parasitic models
- Units (Time / Capacitance / Distance)
- GUI display information (Colors and fill template for layers)
- This file is stored in the design library (Common Database)

The physical library information is stored in the Milkyway design library. For each cell, the Milkyway design library contains several views of the cell, which are used for different physical design tasks. If you have not already created a Milkyway library for your design (by using another tool that uses Milkyway), you need to create one by using the IC Compiler tool. If you already have a Milkyway design library, you must open it before working on your design.

5.4 Setting Up the Power and Ground Nets

IC Compiler uses variables to define names for the power and ground nets. In each session, you must define the values for the following variables (either interactively or in the .synopsys_dc.setup file) so that IC Compiler can identify the power and ground nets:

- `mw_logic0_net`

By default, IC Compiler VSS as the ground net name. If you are using a different name, you must specify the name by setting the `mw_logic0_net` variable.

- `mw_logic1_net`

By default, IC Compiler uses VDD as the power net name. If you are using a different name, you must specify the name by setting the `mw_logic1_net` variable.

5.5 Annotating the Physical Data

IC Compiler provides several methods of annotating physical data on the design:

- Reading the physical data from a DEF file
- Reading the physical data from a floorplan file
- Copying the physical data from another design

5.5.1 Reading DEF Files

To read a DEF file, use the `read_def` command (or choose File > Import > Read DEF in the GUI). To include the physical objects, use the `-allow_physical` option on the command line or select “Allow physical objects” in the GUI.

```
icc_shell> read_def -allow_physical design_name.def
```

By default, the `read_def` command is additive and adds the physical data in the DEF file to the existing physical data in the design.

5.5.2 Reading Floorplan Files

A floorplan file is a file that you previously created by using the `write_floorplan` command. This file is a script that contains information about the floorplan. If the power and ground nets are not explicitly defined in your netlist, you must create the power and

ground connections by using the `connect_pg_nets` command before reading the floorplan file. To read a floorplan file, use the `read_floorplan` command.

```
icc_shell> read_floorplan floorplan_file_name
```

5.5.3 Copying Physical Data

To copy physical data from the layout (CEL) view of one design in the current Milkyway design library to another, use the `copy_floorplan` command. Specify the design from which to copy the physical data by using the `-from` option.

```
icc_shell> copy_floorplan -from design1
```

This command copies the following physical data:

- **General floorplan data** - This includes information such as the design boundary, the core boundary, the placement rows, the placement sites, and the wire tracks.
- **Obstructions** - These include information such as placement blockages (soft and hard), keepout margins (soft and hard), all route guides, and move bounds (soft and hard).
- **Cell instances** - These include fixed, soft-fixed, and placed cells (such as macros, pads, and preplaced standard cells), as well as physical cells (such as corner pads, filler cells, and via cells).
- **Preroutes** - These include power and ground preroutes, clock preroutes, signal preroutes and shieldings.
- **Power net connections**
- **Plan groups**

Macro cell placement (optional)

To copy the macro cell placement information, use the `-macro` option.

I/O pad placement (optional)

To copy the I/O pad placement information, use the `-pad` option.

5.6 Preparing for Timing Analysis and RC Calculation

IC Compiler provides RC calculation technology and timing analysis capabilities for both preroute and postroute data. Before you perform RC calculation and timing analysis, you must complete the following tasks:

- Set up the TLUPlus files

- Set the timing constraints
- Specify the analysis mode
- Select the delay calculation algorithm

5.6.1 Setting up the TLU-Plus Files

TLUPlus is a binary table format that stores the RC coefficients. The TLUPlus models enable accurate RC extraction results by including the effects of width, space, density, and temperature on the resistance coefficients. For details about modeling these effects in the Interconnect Technology Format (ITF) file, see the STAR-RCXT documentation. To use TLUPlus models for RC estimation and extraction, you must specify the following files:

- **The map file** - The map file matches the layer and via names in the Milkyway technology file with the names in the ITF file.
- The maximum TLUPlus model files
- (Optional) the minimum TLUPlus model files

You specify these files by using the `set_tlu_plus_files` command. You can specify TLUPlus file names with or without their full paths. If you do not include the paths, IC Compiler uses the search paths defined with the `search_path` variable to find the files.

```
icc_shell> set_tlu_plus_files \
-tech2itf_map ./path/map_file_name.map \
-max_tluplus ./path/worst_settings.tlup \
-min_tluplus ./path/best_settings.tlup
```

5.6.2 Setting Timing Constraints

Timing Constraints are required to communicate the design's timing intentions to ICC. At a minimum, the timing constraints must contain a clock definition for each clock signal, as well as input and output arrival times for each I/O port. This requirement ensures that all signal paths are constrained for timing. To model the clock tree effects for placement (before running clock tree synthesis), you should also define the following constraints for each clock: `set_clock_uncertainty`, `set_clock_latency`, and

set_clock_transition. You can use a file of Synopsys Design Constraints (SDC) commands or use individual SDC commands. To read a timing constraints file, use the read_sdc command.

```
icc_shell> read_sdc - design_name.sdc
```

The following figure shows the complete overview of the IC Compiler.

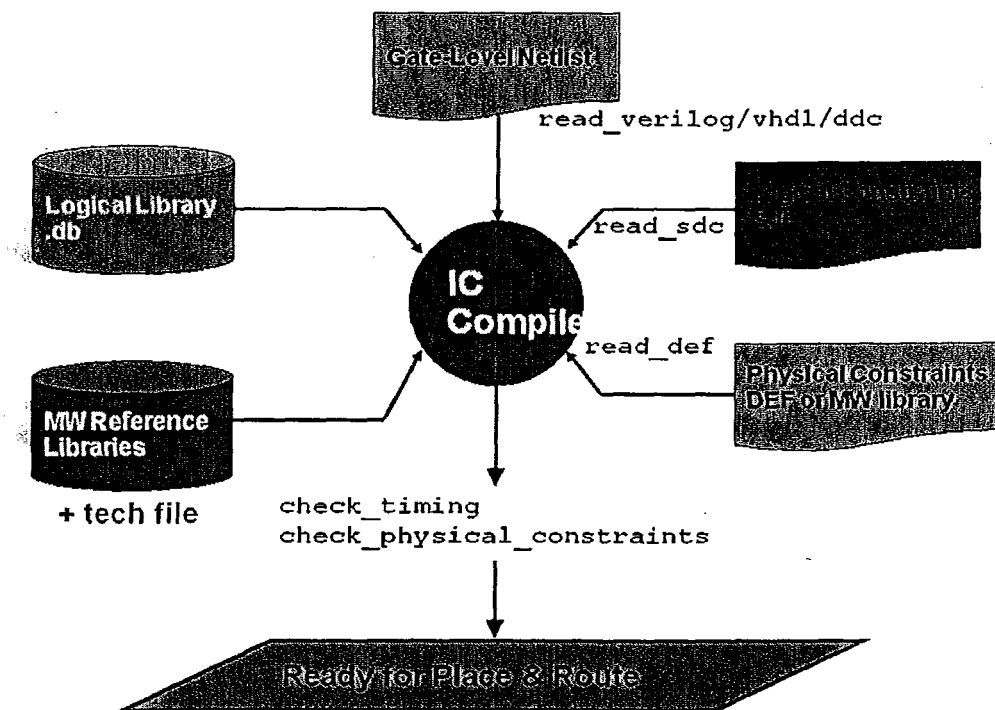


Figure: 5.2 - ICC Overview

5.7 Placement

The placement in VLSI design is the first phase in the process of designing the physical layout of a chip. Since the quality of the attainable routing is to a high degree determined by the placement. In the placement problem we are given a set of rectangular modules (or cells/circuits/macros) of different height and width that should be placed disjointly on the chip surface. Depending on the layout process used, all modules may have the same height (standard cell problem) or variable height (general cell problem). Every module has a number of connection points, so-called pins, and the netlist is a partitioning of the pins into nets that should be interconnected.

Objective function for placing elements is:

- Minimize area
- Routability (minimize wire length)
- Power dissipation should be minimized

The main command during placement in ICC is `place_opt`, this command performs simultaneous placement, routing, and optimization on the current design. The output of this command is a legally placed netlist. The Direct Timing-Driven Placement engine (DTDP) is used in the mainstream `place_opt` flows. DTDP is the first in the industry to incorporate slack directly into the placement objective. Until now, placers were using indirect slack models like increasing the weight/importance of timing-critical nets.

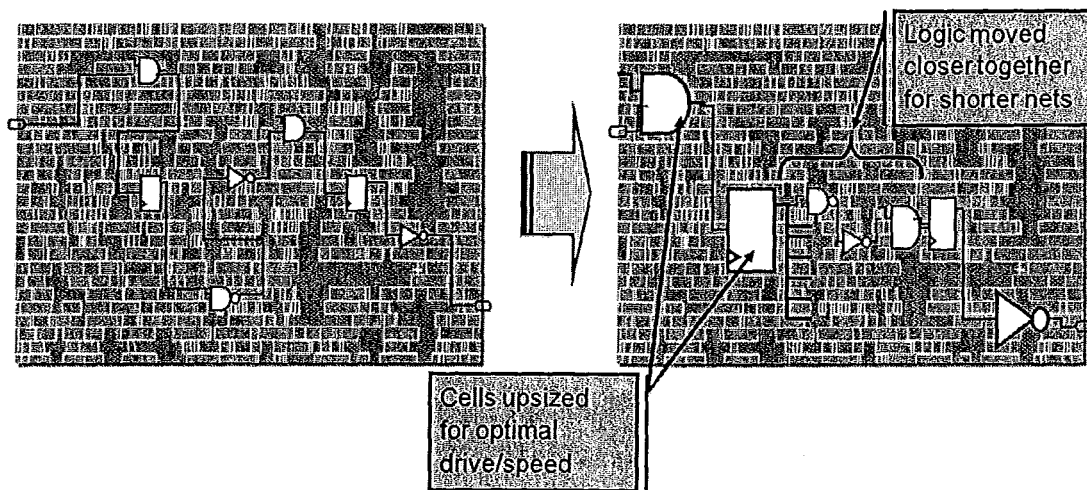


Figure: 5.3 - Iterative placement and optimization

SYNTAX:

```
place_opt
    [-area_recovery]
    [-congestion]
    [-effort]
    [-power_mode none | all | leakage]
```

Since we need a low power design we use “**place_opt – power**”, this command Enables different power optimization in **place_opt**. The power optimization features are determined by the command’s `set_power_options`. By default the tool runs leakage power

optimization when you use the **-power** option. This option can be combined with all other current options to **place_opt**. This includes the **-area_recovery** option and the **-effort (low | medium | high)** option by default, this option is off. Overall the command does Optimize leakage power and dynamic power and perform low-power placement. The power optimizations and low-power placement capability must be previously enabled.

5.8 Clock Tree Synthesis (CTS)

A clock (buffer) tree is built to balance the output loads and minimize the clock skew a delay line can be added to the network to meet the minimum insertion delay (clock balancing), buffers are used to speed up the clock signals.

Effects of CTS:

- Several (Hundreds/Thousands) of clock buffers added to the design
- Placement / Routing congestion may increase
- Non-clock cells may have been moved to less ideal locations
- Timing violations can be introduced

Before running clock tree synthesis, your design should meet the following requirements:

- The design is placed and optimized.

Use the **check_legality -verbose** command to verify that the placement is legal. Running clock tree synthesis on a design that does not have a legal placement might result in long runtimes and reduced QoR.

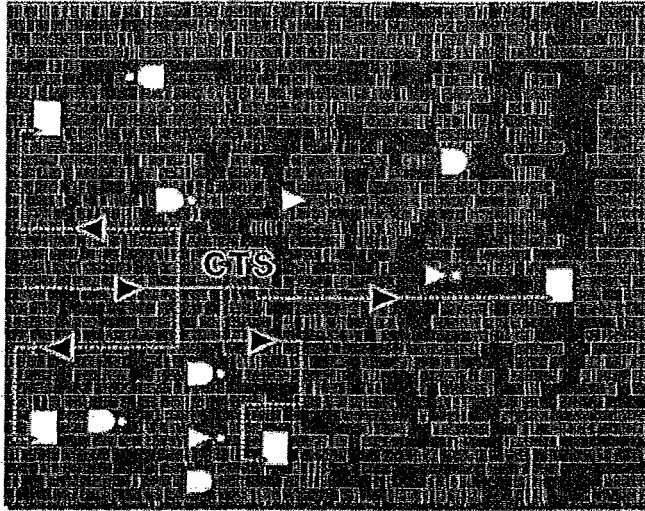


Figure: 5.4 - Clock tree Synthesis

The estimated QoR for the design should meet your requirements before you start clock tree synthesis. This includes acceptable results for – Congestion If congestion issues are not resolved before clock tree synthesis, the addition of clock trees can increase congestion. If the design is congested, you can rerun place_opt with the -congestion and -effort high options, but the runtime can be long.

- Timing
- Maximum capacitance
- Maximum transition time
- The power and ground nets are prerouted
- High-fanout nets, such as scan enables, are synthesized with buffers

5.8.7 Optimizing for Power

When goal is to minimize power, use a large (or unlimited) maximum clock-gating fanout during insertion of the integrated clock gating cells (ICGs). After placing design, perform clock tree power optimization before running clock tree synthesis. To perform clock tree power optimization:

1. Specify the clock tree references
2. Specify the clock tree synthesis options
3. Enable the power optimization techniques by using the set_power_options command:
 - Specify -clock_gating true to enable physical optimization of the integrated clock

gating cells. - Specify `-low_power_placement` true to enable power-aware placement of the integrated clock gating cells and registers.

4. Annotate the switching activity by running the `read_saif` or `set_switching_activity`

If you use a SAIF file to annotate the switching activity, verify that it was generated at the actual clock frequency. If the SAIF file was not generated at the actual clock frequency, the numbers reported for dynamic power will be very small and incorrect.

5. Disable automatic group bounding of the gated clocks by setting the `placer_disable_auto_bound_for_gated_clock` variable to true.

6. (Optional) Specify the design constraints used during clock tree power optimization.

Specify the design constraints used during clock tree power optimization in a Tcl script file. You create separate constraint files for clock tree optimization and for post-clock-tree-synthesis optimization.

7. Perform clock tree power optimization.

To perform clock tree power optimization as part of the `clock_opt` process, use the `-power` option when you run the `clock_opt` command. When you run the `clock_opt -power` command, IC Compiler also performs leakage optimization and dynamic power optimization, if you have enabled these optimizations by using the `set_power_options` command.

IC Compiler performs the following tasks during clock tree power optimization:

- a. Merges integrated clock gating cells that have the same enable signal. Only integrated clock gating cells with positive slack are merged; cells with negative slack are not merged.

- b. Performs high-fanout net synthesis to fix design rule violations on the enable signals.

Handling Specific Design Characteristics

- c. Performs power-aware placement of the integrated clock gating cells and registers. If a significant portion of the clock gates are disabled, power-aware placement is not able to effectively optimize the design.

By default, clock tree power optimization ignores `dont_touch` and `size_only` attributes on integrated clock gating cells. To report the current clock tree power optimization options, use the `report_optimize_pre_cts_power_options` command.

5.8.8 Implementing the Clock Trees

The recommended process for implementing the clock trees in the design is to use the `clock_opt` command, which performs clock tree synthesis, clock tree optimization, and incremental physical optimization. This process results in a timing optimized design with fully implemented clock trees. To perform clock tree synthesis, clock tree optimizations, and incremental physical optimization, use the `clock_opt` command.

By default, the `clock_opt` command uses virtual routing throughout the clock tree synthesis and optimization process to estimate the wire delay and capacitance. For better correlation with postroute timing, you can use the integrated clock global router to estimate the wire delay and capacitance during the clock tree optimization and interclock delay balancing phases. To enable the integrated clock global router for these phases, set the `cts_integrated_global_router` variable to true before running `clock_opt`.

The `clock_opt` command does the following:

1. (Optional) Performs clock tree power optimization.

To perform clock tree power optimization during the `clock_opt` process, enable physical optimization of the integrated clock gating cells and power-aware placement, as described in “Optimizing for Power”, and use the `-power` option of the `clock_opt` command.

2. Synthesizes the clock trees

Before implementing the clock trees, IC Compiler upsizes, and possible moves, the existing clock gates, which can improve the quality of results (QoR) and reduce the number of clock tree levels.

IC Compiler builds clock trees that meet the clock tree design rule constraints, while balancing the loads and minimizing the clock skew. In addition, IC Compiler optimizes the clock paths beyond exclude pins, stop pins, and float pins to fix any design rule constraint violations.

3. Optimizes the clock trees

During clock tree optimization, IC Compiler uses the optimization techniques, such as buffer relocation, buffer sizing, delay insertion, gate sizing, and gate relocation, to further improve the skew.

4. (Optional) Performs interclock delay balancing

To perform interclock delay balancing during the `clock_opt` process, define the interclock delay balancing requirements, and use the `-inter_clock_balance` option of the `clock_opt` command.

5. Performs detail routing of the clock nets

You can also perform detail routing of the clock nets as a stand-alone process (the `route_group -all_clock_nets` command).

6. Performs RC extraction of the clock nets and computes accurate clock arrival times

7. (Optional) Adjusts the I/O timing

To adjust the input and output delay based on the actual clock arrival time use the `-update_clock_latency` option of the `clock_opt` command. IC Compiler uses the adjusted input and output delays during placement and timing optimization.

8. Optimizes the scan chains (Optional)

To optimize the scan chains by reordering the chains to minimize the number of buffer crossings in the scan chain, use the `-optimize_dft` option of the `clock_opt` command.

9. Fixes the placement of the clock tree buffers and inverters

10. Performs placement and timing optimization.

If you specify `-update_clock_latency`, IC Compiler uses the adjusted input and output delays during placement and timing optimization. IC Compiler uses propagated arrival times for all clock sinks. we can customize the placement and timing optimization process by specifying the following options: `-area_recovery`, `-in_place_size_only`, and `-size_only`. we can perform leakage and dynamic optimization

11. Performs power optimization (Optional)

You can perform leakage power optimization and dynamic power optimization during the `clock_opt` process by enabling the selected optimizations with the `set_power_options` command and using the `-power` option of the `clock_opt` command. To enable leakage power optimization, use the `set_power_options -leakage` command. To enable dynamic power optimization, use the `set_power_options -dynamic` command.

12. Fixes hold time violations (Optional)

To fix hold time violations during the clock_opt process, use the -fix_hold_all_clocks option of the clock_opt command.

5.9 Routing

Routing creates physical connections to all clock and signal pins through metal interconnects. Routed paths must meet setup and hold timing, max cap/trans, and clock skew requirement and metal traces must meet physical DRC requirements.

Metal traces (routes) are built along, and centered upon routing tracks based on a grid. Each metal layer has its own grid and preferred routing direction:

- M1: Horizontal
- M2: Vertical, etc...

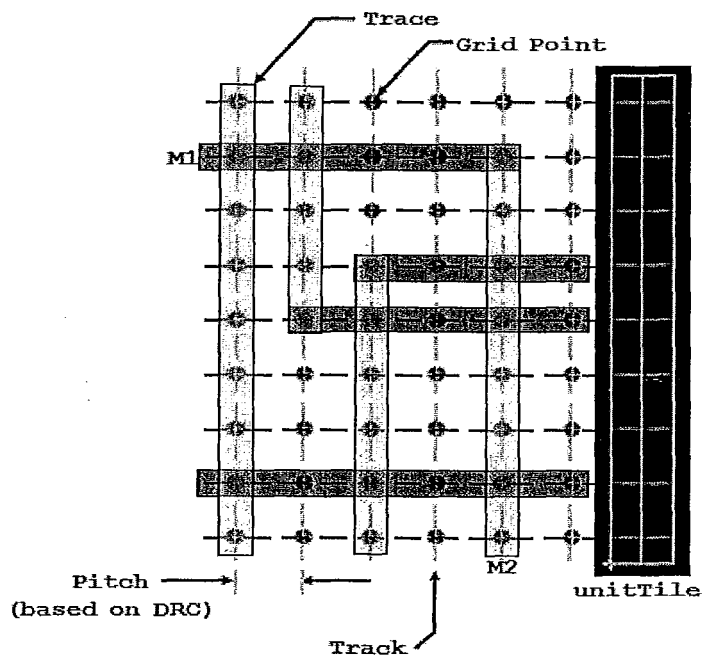


Figure: 5.5 - Grid-Based Routing System

The tracks and preferred routing directions are defined in a "unitTile" cell in the standard cell library. IC Compiler performs:

- Global Routing
- Track Assignment

- Detail Routing
- Search and Repair

5.9.1 Prerequisites for Routing

Before you can perform routing, your design must meet the following conditions:

- Power and ground nets have been routed after design planning and before placement.
- Clock tree synthesis and optimization have been performed.
- Estimated congestion is acceptable.
- Estimated timing is acceptable (about 0 ns of slack).
- Estimated maximum capacitance and transition have no violations.

5.9.2 Routing and Postroute Optimization of Signal Nets

After you have specified your routing options and routing and postroute optimization strategy, you are ready to perform routing and postroute optimization of your design. The routing and postroute optimization command runs global routing in timing-driven mode and track assignment in crosstalk-prevention mode. These modes automatically enable the density-driven mode for global routing and track assignment. To run routing and postroute optimization in signal integrity mode, use the `set_si_options` command.

To perform routing and postroute optimization of signal nets,

1. Choose Route > Core Routing and Optimization.

The “Core Routing and Optimization” dialog box appears.

2. Select an optimization effort level (Low, Medium or High).

By default, IC Compiler uses the medium effort level. Higher effort levels provide more-aggressive optimization at a runtime cost.

3. Specify when to perform optimization.

By default, IC Compiler performs optimization after all the routing steps (global routing, track assignment and detail routing) are completed, but you can have it occur after global routing, track assignment, or detail routing.

4. Specify the signal integrity optimization option.

By default, IC Compiler does not perform signal integrity optimization (“No optimization” radio button). You can perform crosstalk reduction and signal integrity optimization in addition to routing (by selecting the “Crosstalk reduction and SI optimization” radio button) or only crosstalk reduction and signal integrity optimization (by selecting the “Crosstalk reduction optimization only” radio button).

5. Select one or more of the following options, depending on your requirements:

- Perform power optimization
- Skip initial routing – Skips the routing steps and performs only postroute optimization
- Initial routing only – Performs all the routing steps (global routing, track assignment, and detail routing) or individual routing steps, without postroute optimization. The routing steps performed are the ones you select in the “Run optimization after” area.
- Sizing only optimization – Resolves setup timing violations by sizing only.
- Optimize wire and via routing – Reduces wire length and number of vias.
- Recover area for cells that are not on critical timing paths

6. To perform incremental optimization steps, select “Incremental mode.” You can select “All incremental mode optimization steps” or select from the following options to perform individual optimization steps:

- Timing optimization only with wire sizing only
- Hold timing optimization only

You must first enable hold fixing by using the `set_fix_hold` command.

- Area recovery only
- Fix design rules only
- Power recovery only

5.9.3 Global Routing

Global routing maps general pathways through the design for each unrouted net (signal nets and clock nets). The global router uses a two-dimensional array of global routing cells to model the demand and capacity of global routing. IC Compiler uses the average height of the standard cells to create the height and width of each global routing cell. During global routing, IC Compiler assigns nets to the global routing cells through which

they pass. For each global routing cell, the routing capacity is calculated according to the blockages, pins, and routing tracks inside the cell. Although the nets are not assigned to the actual wire tracks during global routing, the number of nets assigned to each global routing cell is noted. IC Compiler calculates the demand for wire tracks in each global routing cell and reports the overflows, which are the number of wire tracks still needed after IC Compiler assigns nets to the available wire tracks in a global routing cell. IC Compiler might reduce overflows by detouring nets around congested areas and increasing the wire length. You can examine the global routing report that appears in the command window and display congestion maps to help you decide whether your design can be routed.

5.9.4 Track Assignment

Before you perform detail routing, run track assignment to specify which tracks within each global routing cell are to be used for each net. Track assignment operates on the entire design at once; it can make long routes straight and reduce the number of vias, whereas the detail routing routes a small area at a time. After track assignment finishes, all nets are routed, but not very carefully. There are many violations, particularly where the routing connects to pins. Detail routing works to correct those violations.

5.9.5 Detail Routing and Search and Repair

Detail routing uses the general pathways suggested by global routing and track assignment to route the nets (paths and contacts). During search-and-repair routing passes, IC Compiler searches for DRC violations and reroutes wires in an effort to avoid violations. IC Compiler does not add metal stubs on frozen nets to fix DRC violations, even when the nets have DRC violations.

5.10 Power Optimization in ICC

When you activate power optimization, IC Compiler can perform the following types of power optimization:

- Low-Power Placement
- Leakage Power Optimization

- **Dynamic Power Optimization**

The default setup for power optimization performs only leakage power optimization during placement or routing. To perform other power optimizations, you must enable each of them separately. You can also change the constraints and other settings that are used for these power optimizations. The following sections explain the set up and use of the power optimizations.

During physical synthesis, the `place_opt` command can concurrently optimize the design for dynamic and leakage power. The `set_power_options` command has different options to select the type of power optimization and effort level to be used by the `place_opt` command. In the later release of ICC, adaptive leakage optimization was introduced to give better leakage results during the `place_opt` process. To enable this feature set the `adaptive_leakage_opto` variable to `true`. This feature provides balanced and correlated power results with simultaneous emphasis on timing, area, and congestion. It also minimizes the usage of higher-leakage, low-Vt cells.

A very important application of instance-based targeting is leakage optimization. Instance-based targeting can be applied selectively on timing critical blocks in the design by specifying multiple-threshold libraries as target libraries for those blocks. For example

5.10.1 Low-Power Placement

Low-power placement reduces capacitance between nets with high switching activity. The reduction in capacitance is achieved by shortening the nets during optimization and placement. To shorten the nets, IC Compiler runs clock tree synthesis to generate a clock tree that helps to determine where best to shorten each net by moving its cells closer together.

To set up for low-power placement,

1. Set the `-low_power_placement` option of the `set_power_options` command to `true`.

```
icc_shell> set_power_options -low_power_placement true
```

To disable low-power placement, set this option to `false`. The default value of the `-low_power_placement` option is `false`.

2. Set up clock tree synthesis just as you do after performing placement and optimization.
3. Annotate switching activity information to the design for the best results or apply the default toggle rate.

To run low-power placement, use the -power option of the place_opt command.

icc_shell> place_opt -power

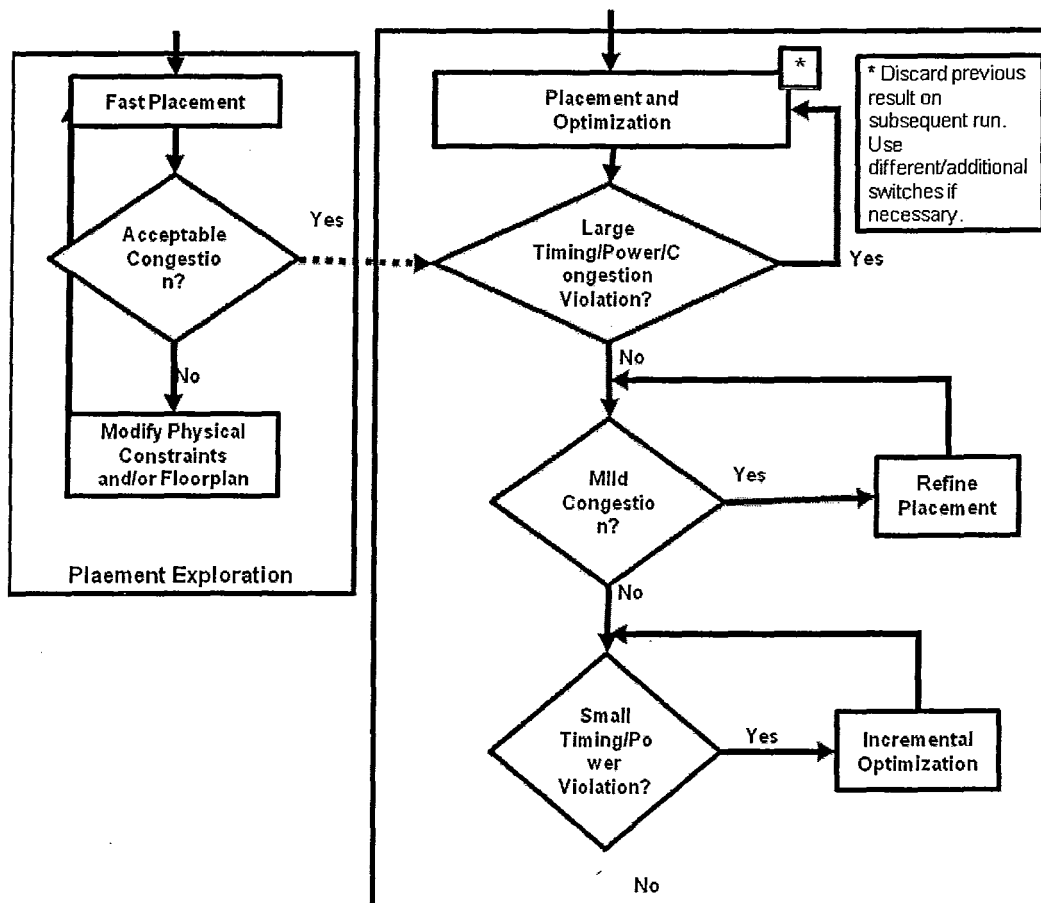


Figure: 5.6 - IC Compiler Placement

5.10.2 Leakage Power Optimization

Leakage power optimization is an additional step to timing optimization that minimizes leakage power without degrading performance. By default, IC Compiler uses an adaptive leakage power optimization algorithm that provides one-pass leakage optimization with minimal impact on timing, area, and congestion. IC Compiler can perform leakage

optimization during placement (`place_opt`), clock tree synthesis (`clock_opt`), and routing (`route_opt`).

Leakage power optimization is enabled by default. To disable leakage power optimization, set the `-leakage` option to false using the `set_power_options` command. You can setup multiple threshold voltage libraries to reduce the leakage power, though leakage optimization works on single threshold libraries also

To run leakage power optimization (and any other power optimizations that you have enabled and that work with placement, clock tree synthesis and routing functions), do one of the following:

- To run placement and optimization, use the `place_opt -power` command.
- To run incremental placement, use the `psynopt -power` command.

```
icc_shell> psynopt -power
```

If you want to perform only power optimization during incremental placement, use the `-power_only` option

```
icc_shell> psynopt -power_only
```

- To run routing and power optimization, use the `route_opt -power` command.

5.10.3 Leakage Power Using Multithreshold Voltage Cells

Leakage power optimization using multi-threshold libraries provides more leakage power savings in the design. The `set_max_leakage_power` constraint causes compile to automatically perform leakage power optimization. The recommended methodology is to use all multiple-Vt libraries for logic synthesis in Design Compiler, and also for physical optimization in IC Compiler. The tool optimizes the design while providing equal emphasis for timing and power and making sure that the number of low-Vt cells used is kept to a minimum. This methodology gives comparable power numbers to that of two-pass leakage flow; meanwhile, timing quality of results is unaffected.

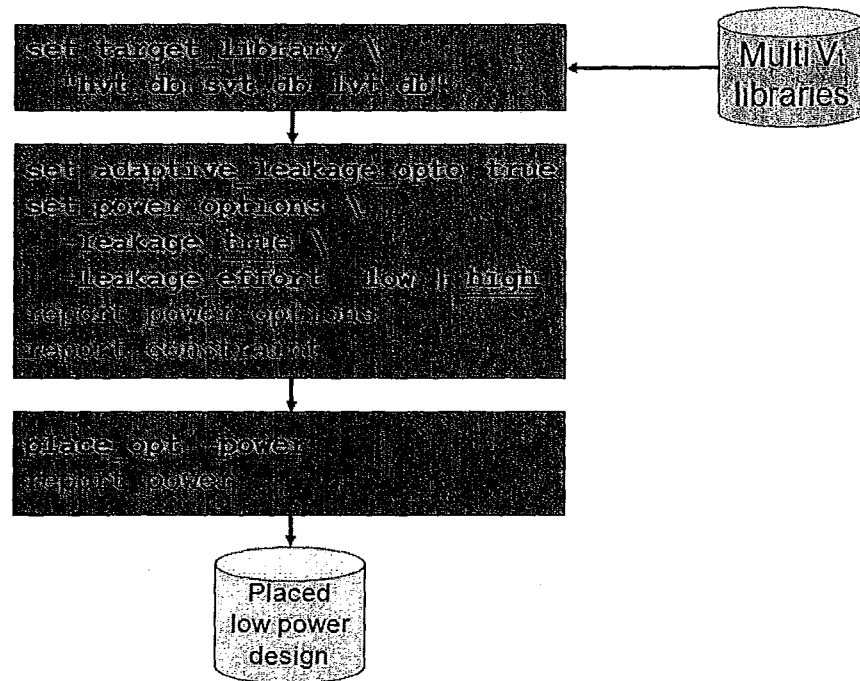


Figure: 5.7 - Usage Model: Leakage Power Optimization

If your target library supports multi threshold voltage cells, you can reduce the leakage power by controlling the percentage of lower threshold voltage cells in your design. Use the `set_max_lvth_percentage` command to specify the maximum percentage of the total design area that can be occupied by lower threshold voltage cells. The optimization commands, `place_opt`, `route_opt` and `clock_opt` when used with the `-only_psyn` option honor the constraint set by the `set_max_lvth_percentage` command. The `max_lvth` specifies the maximum percentage of the total design area that can be occupied by cells belonging to the low voltage threshold group. The value of the `max_lvth` can be between 0 and 100. The `-lvth_groups` option can be used to specify the list of voltage threshold groups that belong to the low voltage threshold group. If you have already set the maximum percentage constraint on the low voltage threshold cells, reset the value by using the `-reset` option.

5.10.4 Dynamic Power Optimization

After performing RTL clock gating or operand isolation, you can use gate-level dynamic power optimization, an additional step to timing optimization, to further reduce the

dynamic power during a placement function. This optimization targets a dynamic power constraint, which is set, by default, to 0 for a well-balanced runtime and quality of results. You can run this optimization when performing placement and optimization or placement alone.

To set up for dynamic power optimization

1. Enable dynamic power optimization.

To enable dynamic power optimization, set the `-dynamic` option to true with `set_power_options` command. By default, dynamic power optimization is disabled.

```
icc_shell> set_power_options -dynamic true
```

2. (Optional) Annotate the switching activity information to the design, unless you want IC Compiler to use the default switching activity.

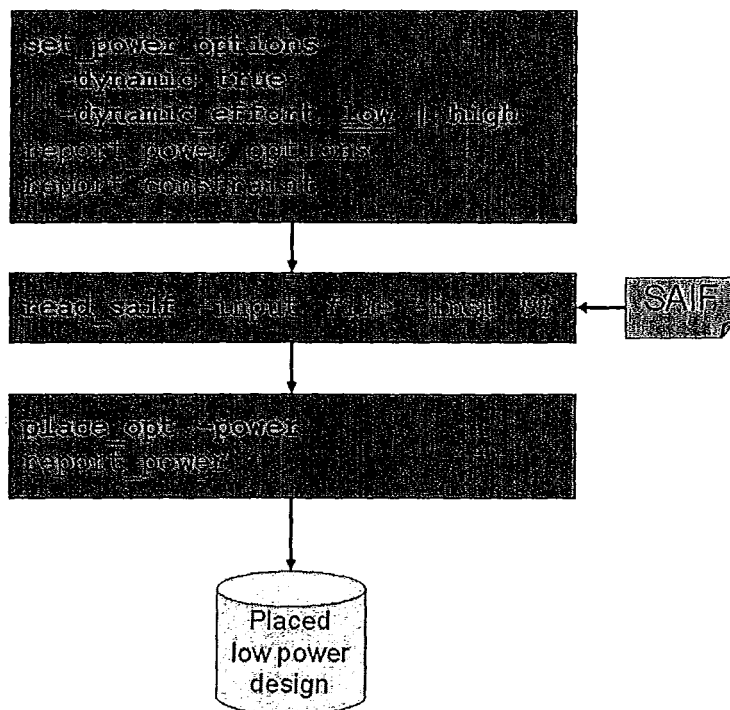


Figure: 5.7 - Usage Model: Dynamic Power Optimization

All power optimizations can be combined, but not all ICC stages can perform all optimizations

	Leakage	LPP	Dynamic
place_opt	✓	✓	✓
psynopt	✓		✓
clock_opt	✓		✓
route_opt	✓		

Table 2 – Stages of power optimization in ICC

Bringing down transistor threshold voltages helps decrease dynamic power but increases sub-threshold leakage current. A power-aware design flow is thus needed to meet timing requirements and keep power consumption within acceptable limits.

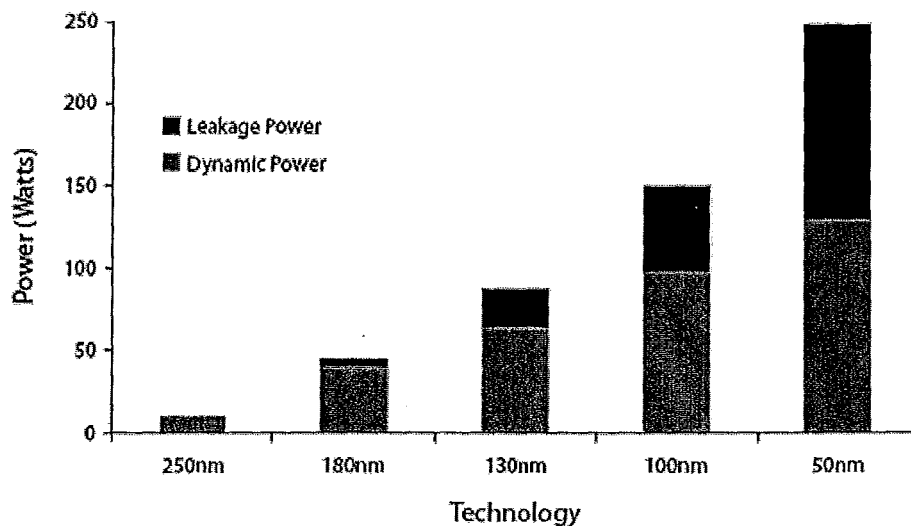


Figure: 5.8 - Leakage Vs Dynamic Power

5.10 Physical Verification

Physical verification checks the design for fabrication feasibility and physical defects that could result in the design to not function properly. Hence there are 3 important checks ICC can perform before tape-out they are as follows:

➤ Design Rule checks (DRC)

Verifies that design does not violate any fabrication rules associated with the target process technology (metal width/space, antenna ratio, etc).

➤ **Electrical Rules Checks (ERC)**

Verifies that there are no short or open circuits with power and ground as well as resistors/capacitors/transistors with floating nodes (part of LVS).

➤ **Layout Versus Schematic (LVS)**

Final physical design matches the logical (schematic) version in terms of correct connectivity and number of electrical devices.

5.12 Fabrication

Physical Design process is complete upon successful completion of timing, functional, and physical verification. The design can be “Taped-Out” and GDSII created for the manufacturer—GDSII (Graphic Design System II) is a binary format containing the physical geometry information of the design. The shapes are assigned numeric attributes in the form of “Layer Number” and “Data Type” (Metal 1 => 100:0). Fabrication and Test determine which chips can be implemented into the system (yield). [13]

CHAPTER: 6 - RISC Architecture

6.1 Features

The OpenRISC 1000 architecture includes the following principal features:

- A linear, 32-bit or 64-bit logical address space with implementation-specific physical address space.
- Simple and uniform-length instruction formats featuring different instruction set extensions:
- OpenRISC Basic Instruction Set (ORBIS32/64) with 32-bit wide instructions aligned on 32-bit boundaries in memory and operating on 32- and 64-bit data
- OpenRISC Vector/DSP eXtension (ORVDX64) with 32-bit wide instructions aligned on 32-bit boundaries in memory and operating on 8-, 16-, 32- and 64-bit data
- OpenRISC Floating-Point eXtension (ORFPX32/64) with 32-bit wide instructions aligned on 32-bit boundaries in memory and operating on 32- and 64-bit data
- Two simple memory addressing modes, whereby memory address is calculated by:
 - addition of a register operand and a signed 16-bit immediate value
 - addition of a register operand and a signed 16-bit immediate value followed by update of the register operand with the calculated effective address
- Two register operands (or one register and a constant) for most instructions who then place the result in a third register
- Shadowed or single 32-entry or narrow 16-entry general purpose register file
- Branch delay slot for keeping the pipeline as full as possible
- Support for separate instruction and data caches/MMUs (Harvard architecture) or for unified instruction and data caches/MMUs (Stanford architecture)
- A flexible architecture definition that allows certain functions to be performed either in hardware or with the assistance of implementation-specific software
- Number of different, separated exceptions simplifying exception model
- Fast context switch support in register set, caches, and MMUs

6.2 Introduction

The OpenRISC 1000 architecture is a completely open architecture. It defines the architecture of a family of open source, RISC microprocessor cores. The OpenRISC 1000 architecture allows for a spectrum of chip and system implementations at a variety of price/performance points for a range of applications. It is a 32/64-bit load and store RISC architecture designed with emphasis on performance, simplicity, low power requirements, scalability and versatility. OpenRISC 1000 architecture targets medium and high performance networking, embedded, automotive and portable computer environments.

Performance features include a full 32/64-bit architecture; vector, DSP and floating-point instructions; powerful virtual memory support; cache coherency; optional SMP and SMT support, and support for fast context switching. The architecture defines several features for networking and embedded computer environments. Most notable are several instruction extensions, a configurable number of general-purpose registers, configurable cache and TLB sizes, dynamic power management support, and space for user-provided instructions. The OpenRISC 1000 architecture is the predecessor of a richer and more powerful next generation of OpenRISC architectures.

6.3 OpenRISC 1200 Architecture Overview

The OR1200 is a 32-bit scalar RISC with Harvard microarchitecture, 5 stage integer pipeline, virtual memory support (MMU) and basic DSP capabilities. Caches are 1-way direct-mapped 8KB data cache and 1-way direct-mapped 8KB instruction cache, each with 16-byte line size. Both caches are physically tagged. By default MMUs are implemented and they are constructed of 64-entry hash based 1-way direct-mapped data TLB and 64-entry hash based 1-way direct-mapped instruction TLB.

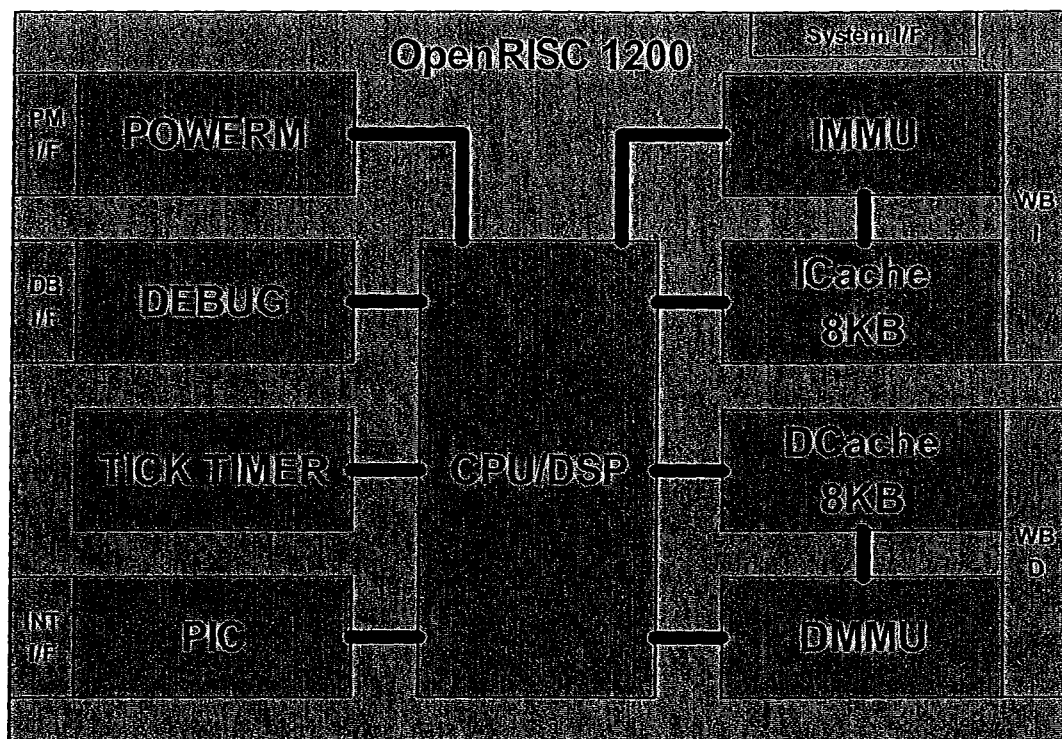


Figure: 6.1 - Core's Architecture

6.3.1 CPU/DSP

CPU/DSP is a central part of the OR1200 RISC processor. Figure: shows basic block diagram of the CPU/DSP. OR1200 CPU/DSP implements only 32-bit part of the OpenRISC 1000 architecture. 64-bit part of the architecture as well as floating-point and vector operations are not implemented in OR1200.

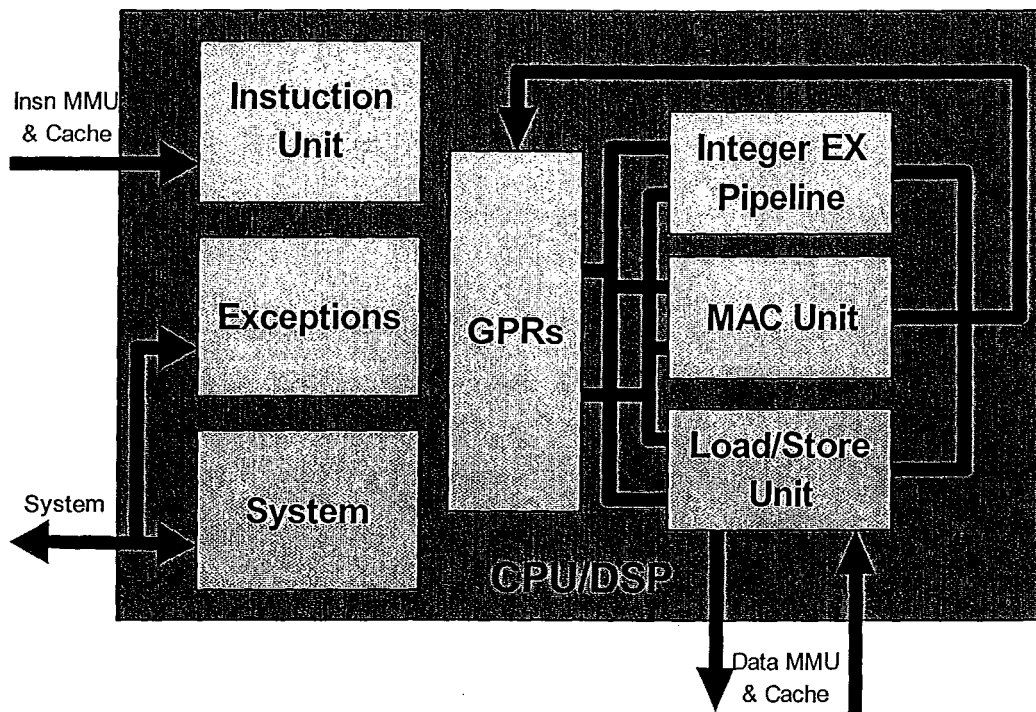


Figure: 6.2 - CPU/DSP Block Diagram

6.3.2 Instruction unit

The instruction unit implements the basic instruction pipeline, fetches instructions from the memory subsystem, dispatches them to available execution units, and maintains a state history to ensure a precise exception model and that operations finish in order. It also executes conditional branch and unconditional jump instructions.

The sequencer can dispatch a sequential instruction on each clock if the appropriate execution unit is available. The execution unit must discern whether source data is available and to ensure that no other instruction is targeting the same destination register. Instruction unit handles only ORBIS32 instruction class. ORFPX32/64 and ORVDX64 instruction classes are not supported by current OR1200. Instruction unit generates instruction fetch effective address and fetches instructions from instruction cache. Each clock cycle one instruction can be fetched. Instruction fetch EA is further translated into physical address by IMMU.

6.3.3 General-Purpose Registers

OpenRISC 1200 implements 32 general-purpose 32-bit registers. OpenRISC 1000 architecture also support shadow copies of register file to implement fast switching between working contexts, however this feature is not implemented in current OR1200 implementation. OR1200 implements general-purpose register file as two synchronous dual-port memories with capacity of 32 words by 32 bits per word. General-purpose register file can supply two read operands each clock cycle and store one result in a destination register. GPRs can be also read and written through development interface.

6.3.4 Load/Store Unit

The load/store unit (LSU) transfers all data between the GPRs and the CPU's internal bus. It is implemented as an independent execution unit so that stalls in memory subsystem only affect master pipeline if there is a data dependency.

The following are LSU's main features:

- all load/store instruction implemented in hardware (atomic instructions included)
- address entry buffer
- pipelined operation
- aligned accesses for fast memory access

When load and store instructions are issued, the LSU determines if all operands are available. These operands include the following:

- address register operand
- source data register operand (for store instructions)
- destination data register operand (for load instructions)

LSU can execute one load instruction every two clock cycles assuming load instruction have a hit in the data cache. Execution of store instructions takes one clock cycle assuming they have a hit in the data cache. LSU performs calculation of the load/store effective address. EA is further translated into physical address by DMMU. Load/store

effective address and load and store data can be also accessed through development interface

6.3.5 Integer Execution Pipeline

The core implements the following types of 32-bit integer instructions:

- Arithmetic instructions
- Compare instructions
- Logical instructions
- Rotate and shift instructions

Most integer instructions can execute in one cycle. For details about timing see table TBD. The core implements the following types of 32-bit integer instructions:

- Arithmetic instructions
- Compare instructions
- Logical instructions
- Rotate and shift instructions

Instruction Group	Clock Cycles to Execute
Arithmetic except Multiply/Divide	1
Multiply	3
Divide	Not implemented
Compare	1
Logical	1
Rotate and Shift	1
Others	1

Table: 6.1 - Execution Time of Integer Instructions

Table: lists execution times for instructions executed by integer execution pipeline. Most instructions are executed in one clock cycle.

6.3.6 MAC Unit

The MAC unit executes DSP MAC operations. MAC operations are 32x32 with 48-bit accumulator. MAC unit is fully pipelined and can accept new MAC operation in each new clock cycle. MAC unit executes l.mac instructions. MAC unit implements 32x32 fully pipelined multiplier and 48-bit accumulator. MAC unit can accept one new l.mac instruction each clock cycle.

6.3.7 System Unit

The system unit connects all other signals of the CPU/DSP that are not connected through instruction and data interfaces. It also implements all system special-purpose registers (e.g. supervisor register). System unit implements system control and status special-purpose registers and executes all l.mtspr/l.mfspr instructions.

6.3.8 Exceptions

Core exceptions can be generated when an exception condition occurs. Exception sources in OR1200 include the following:

- External interrupt request
- Certain memory access condition
- Internal errors, such as an attempt to execute unimplemented opcode
- System call
- Internal exception, such as breakpoint exceptions

Exception handling is transparent to user software and uses the same mechanism to handle all types of exceptions. When an exception is taken, control is transferred to an exception handler at an offset defined by for the type of exception encountered.

Exceptions are handled in supervisor mode. The core implements a precise exception model. This means that when an exception is taken, the following conditions are met:

- Subsequent instructions in program flow are discarded

- Previous instructions finish and write back their results
- The address of faulting instruction is saved in EPCR registers and the machine state is saved to ESR registers

6.4 Register Set

6.4.1 Features

The OpenRISC 1000 register set includes thirty-two or sixteen 32/64-bit general-purpose registers – OpenRISC 1000 implementations optimized for use in FPGAs and ASICs in embedded and similar environments may implement only the first sixteen of the possible thirty-two registers.

6.4.2 General-Purpose Registers (GPRs)

The thirty-two general-purpose registers are labeled R0-R31 and are 32 bits wide in 32-bit implementations and 64 bits wide in 64-bit implementations. They hold scalar integer data, floating-point data, vectors or memory pointers. **Table: 6.2** contains a list of general-purpose registers. The GPRs may be accessed as both source and destination registers by ORBIS, ORVDX and ORFPX instructions.

Register					r31	r30
Register	R29	R28	r27	r26	r25	r24
Register	R23	R22	r21	r20	r19	r18
Register	R17	R16	r15	r14	r13	r12
Register	R11	r10	r9	r8	r7	r6
Register	R5	r4	r3	r2	r1	r0

Table: 6.2 - General-Purpose Registers

R0 is used as a constant zero. Whether or not R0 is actually hardwired to zero is implementation dependent. R0 should never be used as a destination register. An implementation may have several sets of GPRs and use them as shadow registers, switching between them whenever a new exception occurs. The current set is identified

by the SR[CID] value. An implementation is not required to initialize GPRs to zero during the reset procedure. The reset exception handler is responsible for initializing GPRs to zero if that is necessary.

6.5 RISC Design Implementation - Command line flow

6.5.1 Design Compiler flow (Front-End)

Step1. Design Setup

Milkyway related variables

set mw_power_net VDD

set mw_ground_net VSS

set mw_logic1_net VDD

set mw_logic0_net VSS

set mw_power_port VDD

set mw_ground_port VSS

set mw_tech_file /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/astro/tf/cms9flp_8lm_2thick.tf

set mw_reference_library [list \

 /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/astro/cms9flphvt_m_fram \

 /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/astro/cms9flplvt_m_fram \

 /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/astro/cms9flpsvt_m_fram \

]

#DESIGN SETUP

set target_library [list \

 /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro/cms9flphvt_ff_1p1v_m40c.db \

 /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro/cms9flplvt_ff_1p1v_m40c.db \

 /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro/cms9flpsvt_ff_1p1v_m40c.db \

]


```

set link_library [ list {*} \
    /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro_cms9flp-
hvt_ff_1p1v_m40c.db \
    /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro_cms9flp-
lvt_ff_1p1v_m40c.db \
    /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro_cms9flp-
svt_ff_1p1v_m40c.db \
    ]
set physical_library [list \
    /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/astro/plib/
    cms9flphvt_m_8lm_2thick.pdb \
    /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/astro/plib/
    cms9flplvt_m_8lm_2thick.pdb \
    /opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/astro/plib/
    cms9flpsvt_m_8lm_2thick.pdb \
    ]
set time_factor [expr 1.4 * 0.22]

```

Step2. Read design

```

set rtl_files { \
    /alu.v \
    control.v \
    data_path.v \
    instrn_lat.v \
    prgrm_cnt.v \
    prgrm_cnt_top.v \
    prgrm_decode.v \
    prgrm_fsm.v \
    reg_file.v \
    risc_core.v \

```

```

stack_fsm.v \
stack_mem.v \
stack_top.v }
foreach rtl $rtl_files { read_verilog {risc.h $rtl} }
current_design TOP_DESIGN
link

```

Step3. Apply Timing and power constraints

```

set d [all_inputs]
set d [remove_from_collection $d [find port Clk]]
create_clock -period [expr $time_factor * 2.0] [find port Clk]
set_input_delay 0 -clock Clk $d
set_output_delay 0 -clock Clk [all_outputs]
set_max_delay -from $d -to [all_outputs] [expr $time_factor * 2.0]

```

Step4. Compile

```

set_fix_multiple_port_nets -feedthroughs -outputs -buffer_constants
set_clock_gating_style -pos integrated -max 1000000
insert_clock_gating -global
compile

```

Step5. Generate reports

```

report_clock_gating
report_qor
report_timing
report_power

```

Step6. Output

```

#Write out synthesized mapped Gate level netlist to ddc format
write -format ddc -hierarchy -output design.ddc

```

6.5.2 IC Compiler command line flow

Step1. Design setup

The design setup step is common for both Design compiler and IC compiler

Step2. Library Preparation

```
#TO CREATE MILKYWAY LIBRARY
sh rm -rf RISC_die.mw
create_mw_lib -tech
/opt/edalab/snps_libs/DATABASE_SAED90NM_16_July_2008/process/astro/tech/astro
TechFile.tf\
    -mw_reference_library {\
/opt/edalab/snps_libs/DATABASE_SAED90NM_16_July_2008/process/astro/fram/saed
90nm_fr\
    } RISC_die.mw
open_mw_lib design.mw
#TLU plus
set_tlu_plus_files \
    -max_tluplus
/opt/edalab/technology_files/IBM_CMOS/90LP/CMOS9LP_Astro_TLU/AstroTLU_cmo
s9lp_20080218/synopsys/v.20080218/tluplus/cmos9lp_9LB_6_02_00_00_nm.tluplus \
    -tech2itf_map
/opt/edalab/technology_files/IBM_CMOS/90LP/CMOS9LP_Astro_TLU/AstroTLU_cmo
s9lp_20080218/
```

Step3. Read Netlist

```
read_file -format ddc {design.ddc}
link
```

Step4. Floorplan creation

```
initialize_floorplan -start_first_row -flip_first_row
```

Step5. Placement

place_opt
report_timing
report_power

Step6. Clock Tree Synthesis

clock_opt
report_timing
report_power

Step7. Routing

route_opt
report_timing
report_power

Step8. Output – write out to milkyway format

current_design top
save_mw_cel -as top -overwrite

6.6 Results

6.6.1 Analysis

The RISC core module has been synthesized to IBM-ARM's 0.90 μm technology (CMS9FLP) for operation at a nominal supply voltage V_{DD} of 1.1V. Performance has been given highest priority and, thus, the primary objective has been to minimize the clock period. Following the common standards for RISC implementations, the module has been prepared for gated clocks which has been enabled for dynamic power reduction

6.6.2 Timing and power characteristics

The results of the timing-driven synthesis subject to the timing constraints both before and after place and route are summarized in Table 6.2. Obviously, the power consumption of the core module has been underestimated in pre-layout gate-level power analysis. This is due to the shortcomings of wire-load-model-based interconnect capacitance estimation. Also the number of driver cells in the clock network has almost doubled during clock tree synthesis. This leads to a great difference between the results of pre- and post-layout analysis of the power consumption in the clock network.

	Pre-layout analysis	Post-layout analysis
Total Dynamic Power(W)	1.5582 mW	1.8117 mW
Cell Leakage Power(W)	619.3796 nW	944.9301 nW
Clock Period	1ns	
Area (comb./FF)	6789/ 4917	
voltage	1.1v	

Table: 6.3 - Timing and power consumption before and after place & route

6.6.3 Power Optimization potential with clock gating

The effectiveness of the clock gating strategy for power reduction can be seen from Table 6.4. Gated clocks enabled reduce the dynamic power by 78% when compared to dynamic power without clock gating.

Clock Gating	enabled	disabled
Total Dynamic Power(W)	4.8384 mW	1.5582 mW
Cell Leakage Power(W)	731.0982 nW	619.3796 nW

Table: 6.4 - Power consumption before and after clock gating

6.7 Conclusion

The main objectives of this project were to implement RTL to GDSII flow and a way of optimizing the dynamic power consumption of standard-cell-based ASICs by means of clock gating and other power optimization techniques using Synopsys Design compiler

and IC compiler and to investigate the potential of this approach. The methodology developed in this work had to be fully automated in order to minimize the design time.

REFERENCES

- [1] B. Moyer, "Low-power design for embedded processors," Proc. IEEE, vol. 89, pp. 1576-1586.
- [2] D. Mlynek, "Design of VLSI systems," EPFL Integrated Systems Laboratory, <<><http://lsiwww.epfl.ch/LSI2001/teaching/webcourse/>>>, Retrieved Jan 4, 2001.
- [3] "Low power design techniques," Application Note 80, QuickLogic Corporation, 2004.
- [4] R. Kakerow, "Low power design methodologies for mobile communication," Proc. IEEE International Conference on Computer Design, 2002.
- [5] A. Chandrakasan, S. Sheng, and R. W. Brodersen, "Low-power CMOS digital design," IEEE J. Solid State Circuits, pp. 473-484, April 1992.
- [6] P.J.M. Havinga and G.J.M. Smit, "Low power system design techniques for mobile computers," *CTIT technical report series 97-32*, Enschede, Netherlands, 1997.
- [7] "Low power design with CoolRunner-II CPLDs," *Application Note XAPP377*, Xilinx Corporation, 2002.
- [8] M. Khan, "Power optimization in FPGA designs," Altera Corporation, *SNUG San Jose*, 2006.
- [9] <http://www.eetimes.com/news/design/showArticle.jhtml?articleID=1650136>,
Retrieved
Nov 10, 2008.
- [10] David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface (2nd Edition), Morgan Kaufmann Publishers, Inc., 1998,
ISBN 1-55860-428-6 and ISBN 1-55860-491-X.
- [11] "Advanced ASIC chip Synthesis ," Using Synopsys Design Compiler, Physical Compiler and Primetime, Himanshu Bhatnagar.
- [12] Kaijian Shi, Jason Binney, "Design Optimization Methodologies for Low-Leakage Power Designs in Sub- 90nm Technology", Euro DesignCon 2004.
- [13] Simulation and Synthesis Techniques for Asynchronous FIFO Design," Clifford E. Cummings, Sunburst Design, Inc.

- [14] <https://solvnet.synopsys.com>
- [15] <http://www.snug.com>
- [16] SYNOPSYS IC Compiler User Guide [2008]
- [17] SYNOPSYS Design Compiler User Guide [2008]
- [18] TIWARI, ANURAG, “Low Power Fpga Design Techniques for Embedded Systems,”
University of Cincinnati, Engineering: Computer Science & Engineering, 2005.
- [19] Torsten Mahnke, “Low Power ASIC Design Using Voltage Scaling at the Logic Level,” Technische Universität München, 2003.
- [20] Dimitrios Soudris, Christian Piguet, Costas Goutis, “Designing CMOS Circuits for Low Power”, Springer, 2002.
- [21] http://lowpowerdesign.blogspot.com/2008/01/low-power-design-low-power-design_13.html, *Retrieved Nov 10, 2008*.
- [22] ICCAD-2001: IEEE/ACM International Conference on Computer-Aided Design: November 4-8, 2001, Double Tree Hotel, San Jose, CA by IEEE, IEEE Circuits and Systems Society, Published by IEEE, 2001
- [23] BILL MOYER, “Low-Power Design for Embedded Processors,” by IEEE.
- [24] <http://www.opencore.org>, “openRISC 1000”, wishbone interface, 2005
- [25] Ashwin Balakrishnan, “An Experimental Study of the Accuracy of Multiple Power Estimation Methods,” The University of Tennessee, Knoxville, August 2004.

APPENDIX A

Schematic and Layout Views of RISC design

A.1 Design Plan

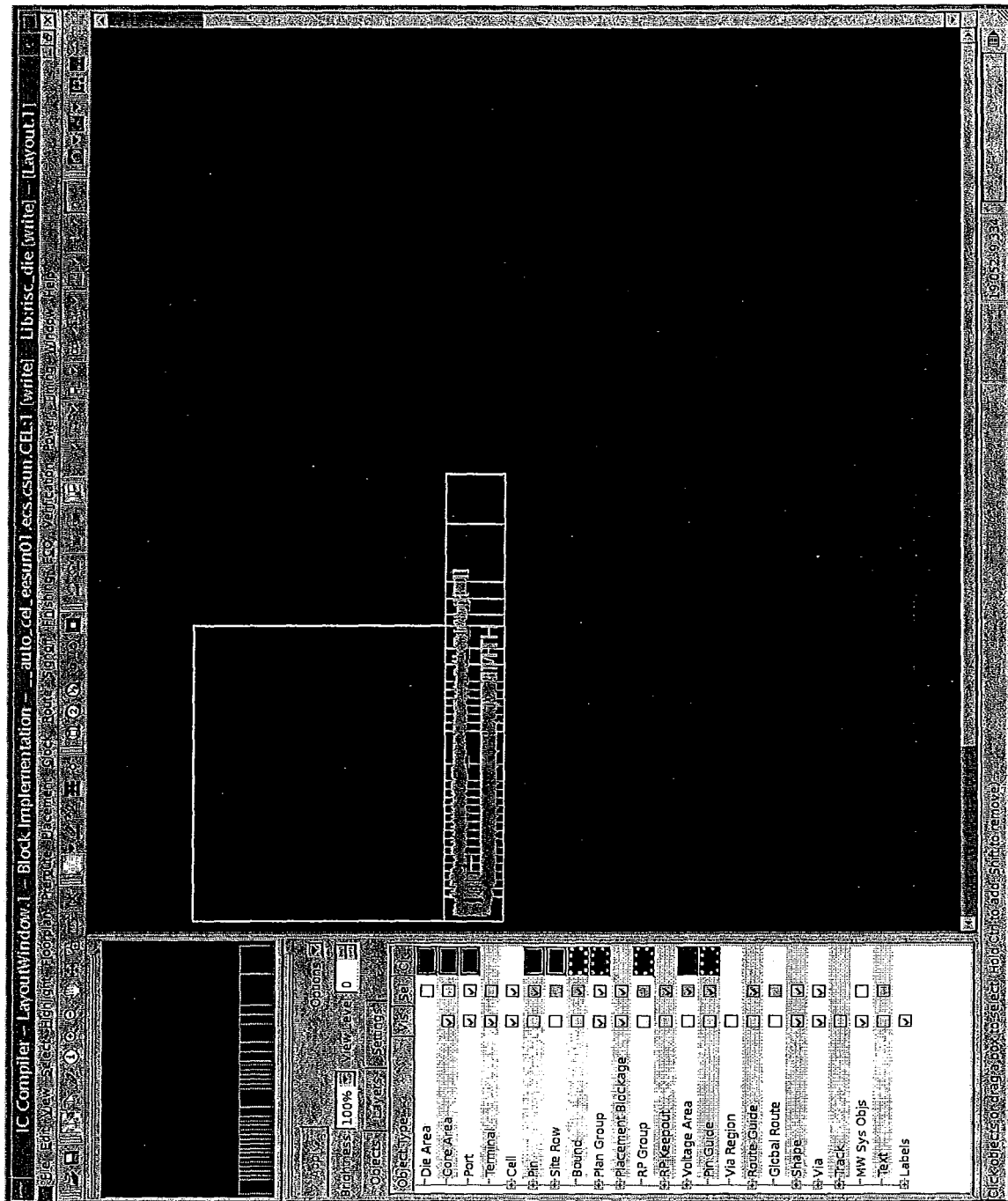


Figure: A.1 - Layout view before Floor plan

A.2 FloorPlan

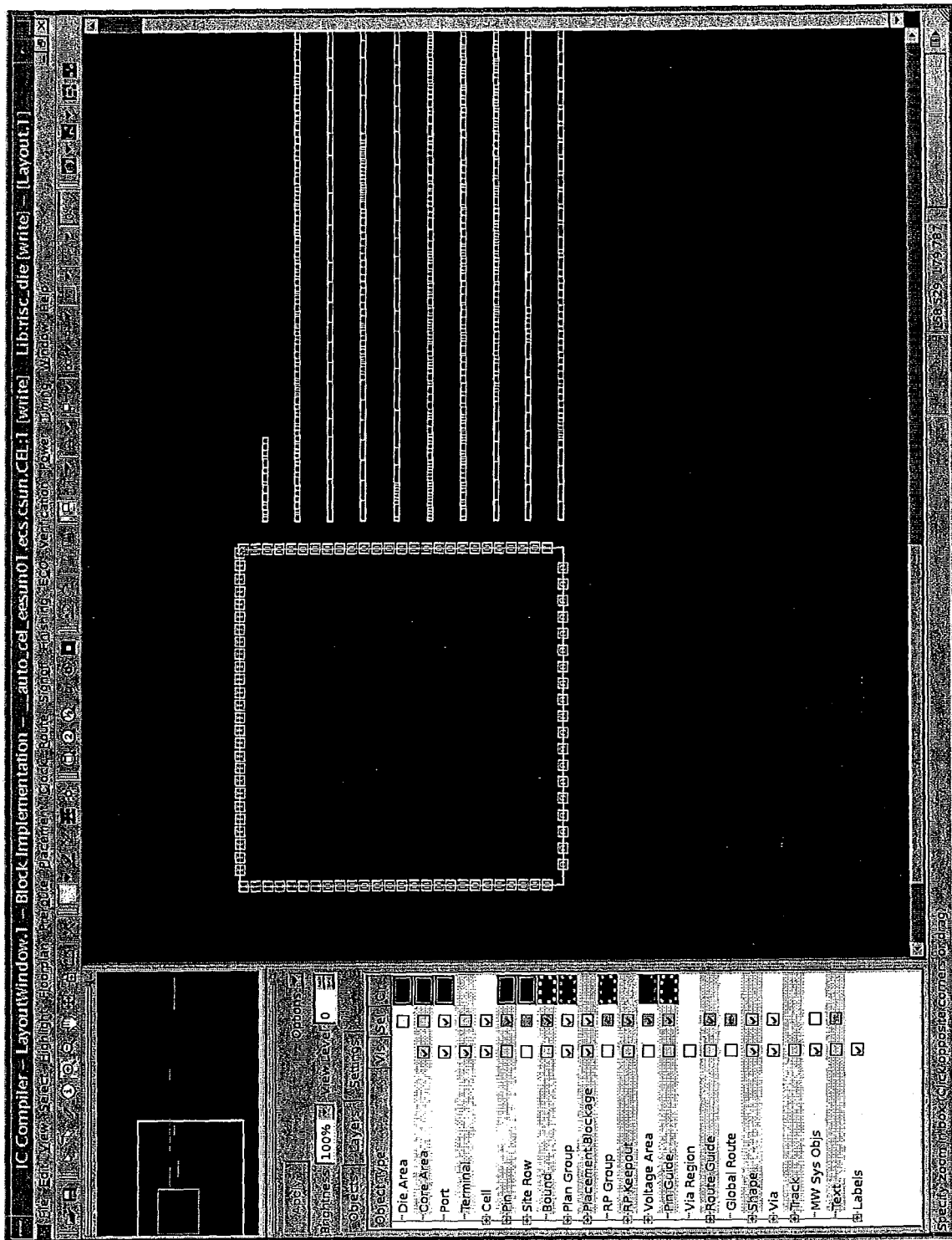


Figure: A.2 - Layout view after Floor plan

A.3 Standard cell Placement

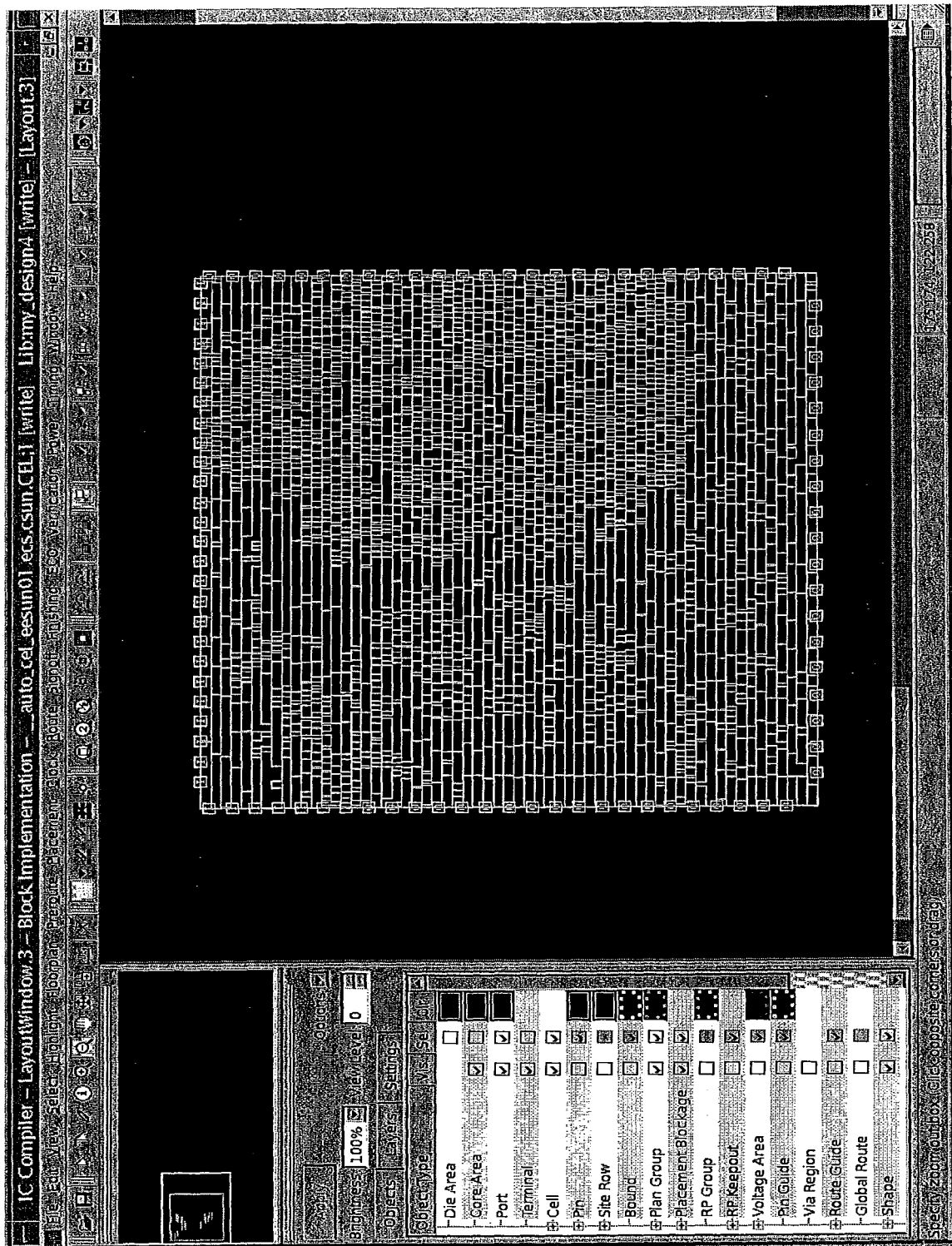


Figure: A.3 – Layout view after placement

A.3.1 Critical path analysis

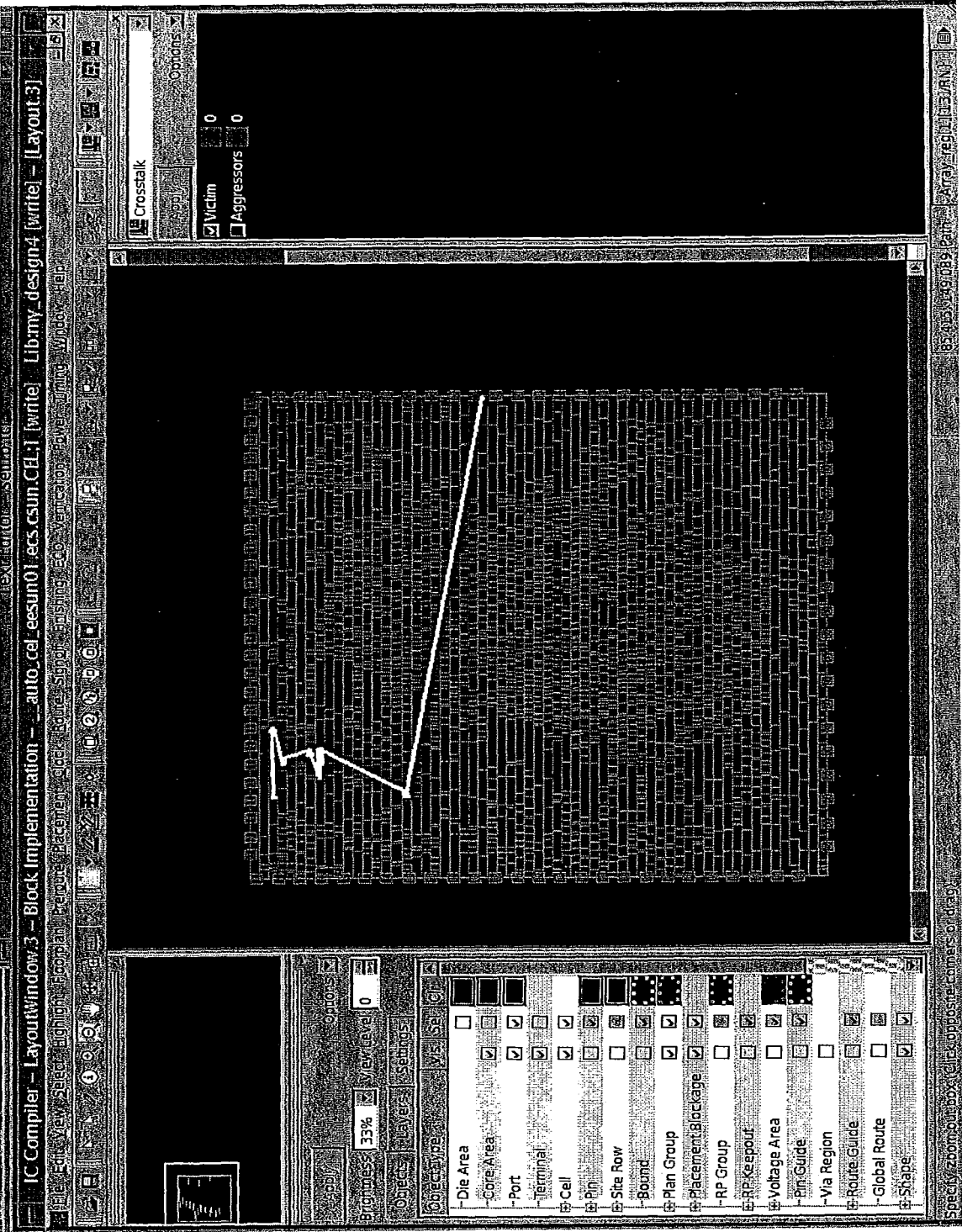


Figure: A.4 - Layout view after placement with critical path

A.4 Routing

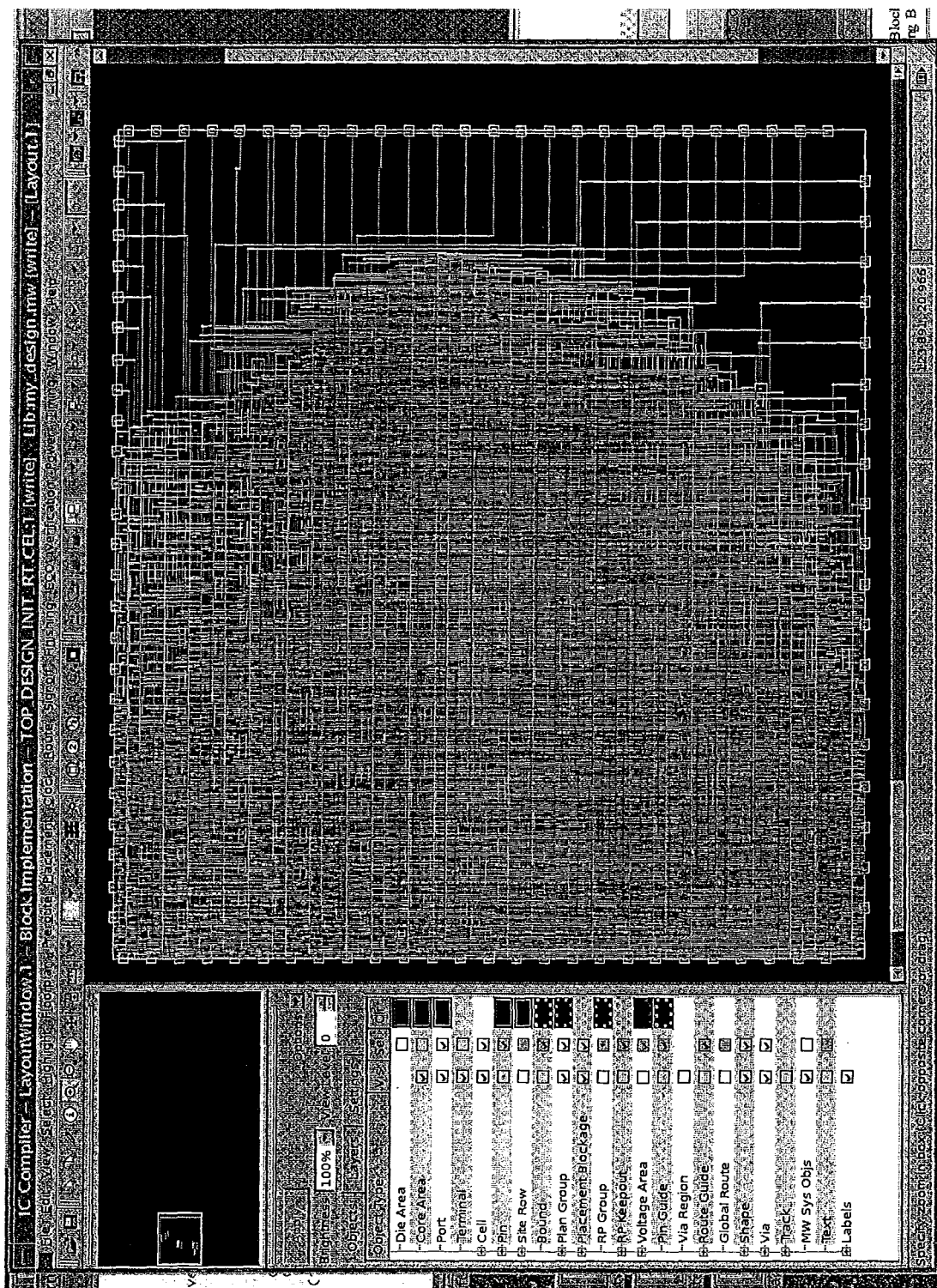


Figure A.5 - Standard cell view after routing

A.5 RTL Schematic

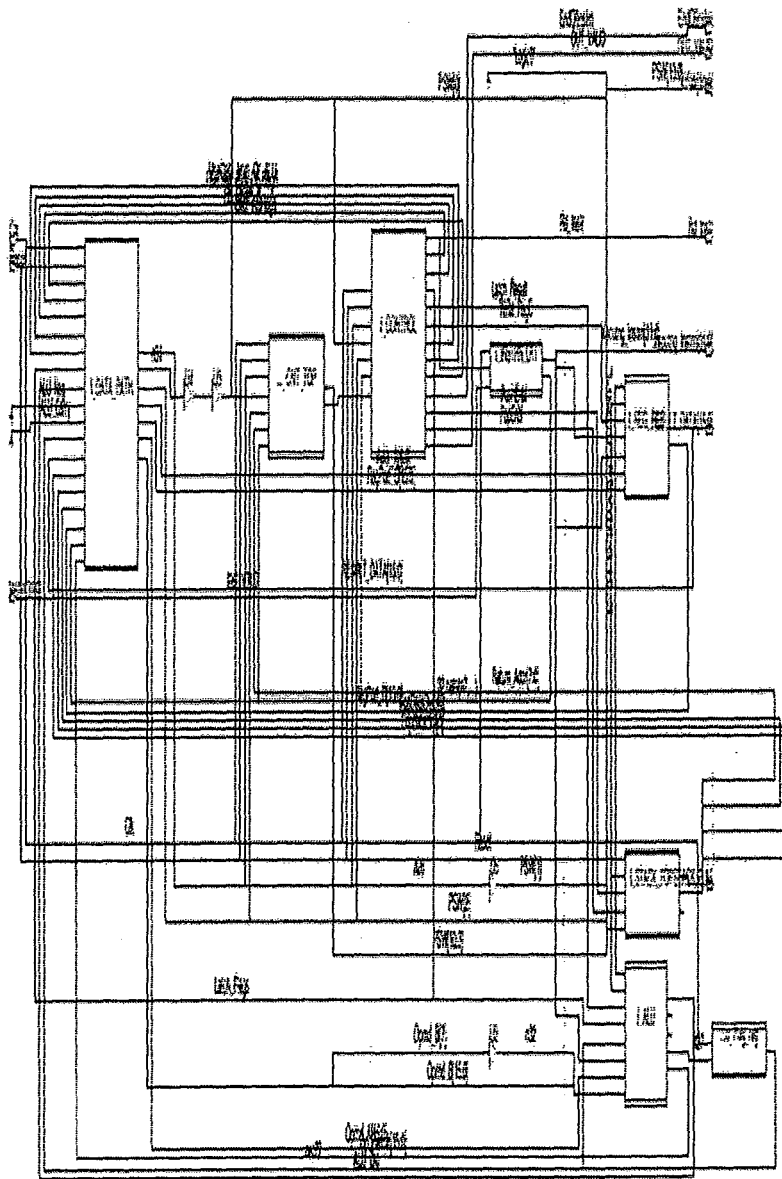


Figure: A.6 - Schematic view of RISC core design in DC

APPENDIX B

Timing, Power and QoR Reports

Report : qor

Design : TOP_DESIGN

Version: A-2007.12-SP5

Date : Sun Nov 30 22:07:41 2008

Timing Path Group 'Clk'

Levels of Logic: 16.00
Critical Path Length: 0.77
Critical Path Slack: -0.22
Critical Path Clk Period: 0.62
Total Negative Slack: -21.05
No. of Violating Paths: 198.00
No. of Hold Violations: 46.00

Cell Count

Hierarchical Cell Count: 40
Hierarchical Port Count: 1006
Leaf Cell Count: 2017

Area

Combinational Area: 6789.204740
Noncombinational Area: 4917.796819

Cell Area: 11707.001559
Design Area: 11707.001559

Design Rules

Total Number of Nets: 2268

Hostname: eesun01.ecs.csun.edu
Compile CPU Statistics

Resource Sharing: 5.44
Logic Optimization: 71.23
Mapping Optimization: 444.83

Overall Compile Time: 572.18

Report : clock_gating
 Design : TOP_DESIGN
 Version: A-2007.12-SP5
 Date : Sun Nov 30 22:07:03 2008

Clock Gating Summary

Number of Clock gating elements	19
Number of Gated registers	296 (96.10%)
Number of Ungated registers	12 (3.90%)
Total number of registers	308

dc_shell> report_timing
 Information: Updating design information... (UID-85)

Report : timing

-path full
 -delay max
 -max_paths 1

Design : TOP_DESIGN
 Version: A-2007.12-SP5
 Date : Sun Nov 30 22:26:45 2008

Operating Conditions: ff_1p1v_m40c Library: scmetro_cms9flp-hvt_ff_1p1v_m40c
 Wire Load Model Mode: top

Startpoint: I_INSTRN_LAT/Crnt_Instrn_2_reg[31]
 (rising edge-triggered flip-flop clocked by Clk)
 Endpoint: I_PRGRM_CNT_TOP/I_PRGRM_CNT/clk_gate_PCint_reg/latch
 (negative level-sensitive latch clocked by Clk)
 Path Group: Clk
 Path Type: max

Point	Incr	Path
clock Clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
I_INSTRN_LAT/Crnt_Instrn_2_reg[31]/CK (DFFRHQX8MTH)	0.00	0.00 r
I_INSTRN_LAT/Crnt_Instrn_2_reg[31]/Q (DFFRHQX8MTH)	0.10	0.10 f

I_INSTRN_LAT/Crnt_Instrn_2[31] (INSTRN_LAT)	0.00	0.10 f	
I_DATA_PATH/Crnt_Instrn[31] (DATA_PATH)	0.00	0.10 f	
I_DATA_PATH/U5/Y (NOR2X4MTH)	0.04	0.14 r	
I_DATA_PATH/U6/Y (AND4X6MTH)	0.09	0.23 r	
I_DATA_PATH/U12/Y (MXI2X2MTH)	0.03	0.26 f	
I_DATA_PATH/Carry_Flag (DATA_PATH)	0.00	0.26 f	
U4/Y (INVX3MTH)	0.03	0.29 r	
U3/Y (CLKINVX6MTH)	0.03	0.31 f	
I_PRGRM_CNT_TOP/Carry_Flag (PRGRM_CNT_TOP)	0.00	0.31 f	
I_PRGRM_CNT_TOP/I_PRGRM_DECODE/Carry_Flag (PRGRM_DECODE)	0.00	0.31 f	
I_PRGRM_CNT_TOP/I_PRGRM_DECODE/U5/Y (XNOR2X2MTH)	0.04		
0.36 f			
I_PRGRM_CNT_TOP/I_PRGRM_DECODE/U29/Y (NOR2X2MTH)	0.04		
0.40 r			
I_PRGRM_CNT_TOP/I_PRGRM_DECODE/U18/Y (MXI2X1MTH)	0.04	0.44	
f			
I_PRGRM_CNT_TOP/I_PRGRM_DECODE/U17/Y (NOR2X2MTH)	0.04		
0.48 r			
I_PRGRM_CNT_TOP/I_PRGRM_DECODE/U12/Y (OAI211X2MTH)	0.05		
0.54 f			
I_PRGRM_CNT_TOP/I_PRGRM_DECODE/U4/Y (NOR3BX4MTH)	0.07		
0.60 f			
I_PRGRM_CNT_TOP/I_PRGRM_DECODE/Incrmnt_PC (PRGRM_DECODE)	0.00	0.60 f	
I_PRGRM_CNT_TOP/I_PRGRM_CNT/Incrmnt_PC (PRGRM_CNT)	0.00	0.60	
f			
I_PRGRM_CNT_TOP/I_PRGRM_CNT/U23/Y (NOR4BXLMTMTH)	0.12	0.72 r	
I_PRGRM_CNT_TOP/I_PRGRM_CNT/U4/Y (BUFX5MTH)	0.08	0.80 r	
I_PRGRM_CNT_TOP/I_PRGRM_CNT/U5/Y (OR4X4MTH)	0.05	0.85 r	
I_PRGRM_CNT_TOP/I_PRGRM_CNT/clk_gate_PCint_reg/EN			
(SNPS_CLOCK_GATE_HIGH_TOP_DE			
	0.00	0.85 r	
I_PRGRM_CNT_TOP/I_PRGRM_CNT/clk_gate_PCint_reg/latch/D (TLATNX1MTH)	0.00	0.85 r	
data arrival time		0.85	
clock Clk (fall edge)	0.46	0.46	
clock network delay (ideal)	0.00	0.46	
I_PRGRM_CNT_TOP/I_PRGRM_CNT/clk_gate_PCint_reg/latch/GN			
(TLATNX1MTH)	0.00	0.46 f	
time borrowed from endpoint	0.39	0.85	
data required time		0.85	

data required time	0.85
data arrival time	-0.85

slack (MET)	0.00

Time Borrowing Information

Clk pulse width	0.46
library setup time	-0.03

max time borrow	0.43
actual time borrow	0.39

```
dc_shell> report_power
Loading db file '/opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro_c
ms9flp-hvt_ff_1p1v_m40c.db'
Loading db file '/opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro_c
ms9flp-lvt_ff_1p1v_m40c.db'
Loading db file '/opt/edalab/9LP_Cell_Pad_Memory_Kit/aci/sc-m/synopsys/scmetro_c
ms9flp-svt_ff_1p1v_m40c.db'
Information: Propagating switching activity (low effort zero delay simulation).
(PWR-6)
Warning: Design has unannotated primary inputs. (PWR-414)
Warning: Design has unannotated sequential cell outputs. (PWR-415)
```

```
*****
Report : power
        -analysis_effort low
Design : TOP_DESIGN
Version: A-2007.12-SP5
Date   : Sun Nov 30 22:27:34 2008
*****
Library(s) Used:
```

```
    scmetro_cms9flp-svt_ff_1p1v_m40c (File: /opt/edalab/9LP_Cell_Pad_Memory_Kit/
aci/sc-m/synopsys/scmetro_cms9flp-svt_ff_1p1v_m40c.db)
    scmetro_cms9flp-lvt_ff_1p1v_m40c (File: /opt/edalab/9LP_Cell_Pad_Memory_Kit/
aci/sc-m/synopsys/scmetro_cms9flp-lvt_ff_1p1v_m40c.db)
    scmetro_cms9flp-hvt_ff_1p1v_m40c (File: /opt/edalab/9LP_Cell_Pad_Memory_Kit/
aci/sc-m/synopsys/scmetro_cms9flp-hvt_ff_1p1v_m40c.db)
```

```
Operating Conditions: ff_1p1v_m40c  Library: scmetro_cms9flp-hvt_ff_1p1v_m40c
Wire Load Model Mode: top
```

Global Operating Voltage = 1.1

Power-specific unit information :

Voltage Units = 1V

Capacitance Units = 1.000000pf

Time Units = 1ns

Dynamic Power Units = 1mW (derived from V,C,T units)

Leakage Power Units = 1pW

Cell Internal Power = 1.3663 mW (88%)

Net Switching Power = 191.8487 uW (12%)

Total Dynamic Power = 1.5582 mW (100%)

Cell Leakage Power = 619.3796 nW